DynaGraph: Holistic Optimization for Latency-Critical GraphRAG Serving via Bidirectional Feedback

ABSTRACT

Graph Retrieval-Augmented Generation (GraphRAG) has emerged as a promising paradigm for complex multi-hop reasoning, bridging knowledge graphs with large language models to answer questions requiring chained evidence. However, deploying GraphRAG in latency-critical serving systems faces a fundamental challenge—the *phase-silo design*: retrievers maximize recall without cost awareness, while serving engines optimize throughput without accounting for query difficulty. This mismatch exacerbates tail latency and SLO violations, especially under skewed workloads. To address this issue, we propose *DynaGraph*, a unified framework that jointly optimizes retrieval, aggregation, and scheduling via bidirectional feedback. It (a) exposes graph-topology features as cost priors, (b) adapts the evidence budget under real-time backpressure, and (c) employs an SLO-aware scheduler that maximizes a newly proposed *QA-Goodput* metric that balances answer quality and timeliness. Experiments on three multi-hop QA benchmarks show that DynaGraph improves QA-Goodput by 1.9–3.0× and reduces median TTFT by 38–51% over the strongest baseline, effectively reconciling deep reasoning with interactive latency.

1 INTRODUCTION

Online question answering (QA) is rapidly evolving from offline benchmarks to interactive, latency-critical deployments, where systems must answer user queries in real time using large knowledge sources and language models [31]. Unlike offline evaluation, where latency is largely irrelevant, online QA must deliver responses within strict time budgets while preserving answer quality.

Consider an *incident-response copilot* used by Site Reliability Engineers (SREs) during a production outage [4]; an engineer might ask: *"Which customer-facing endpoints are affected by Service X's failure, and what mitigation steps were applied in similar past incidents?"* Answering this query requires multi-hop reasoning over heterogeneous enterprise artifacts: service dependency graphs, runbooks, and historical incident summaries [11]. Crucially, the system must adhere to a strict *Time-To-First-Token* ((TTFT)) budget, *i.e.,* the latency between issuing a request and receiving the first token of the response. In such high-stakes settings, answers must be not only correct but also timely, adhering to a *Service Level Objective* (SLO), a quantitative latency target such as TTFT ≤ 2 s for 99% of requests. A correct answer that arrives late violates this contract and is operationally ineffective, as it blocks the human-in-the-loop feedback cycle in which engineers iteratively refine their diagnostic queries based on prior responses.

Example 1: We use an e-commerce customer-support scenario as RE-1: a user reports "payment succeeded but order failed." The copilot must identify affected users, determine the root cause linking payment, order, and inventory events, and recommend fixes based on similar past incidents. Answering this query requires multi-hop reasoning that connects heterogeneous evidence sources (*e.g.,* transaction records, order-service failure traces, and historical tickets and runbooks), by traversing and aggregating distributed evidence under strict latency constraints. □

To address the complexity of such queries, existing research has evolved along two largely orthogonal optimization paths, as summarized in Figure 1. On *the retrieval side*, GraphRAG paradigms [14, 32] and neuro-symbolic approaches (*e.g.,* HippoRAG [18]) capture long dependency chains by constructing dense subgraphs or leveraging Personalized PageRank. These methods enhance reasoning depth and reduce hallucinations by incorporating richer structural context. However, they primarily optimize retrieval recall (*i.e.,* the fraction of relevant evidence successfully retrieved) and semantic completeness, with limited awareness of downstream execution cost. Consequently, they may retrieve large volumes of context, (*e.g.,* dense neighborhoods around "supernode" entities), inflating prefill latency and increasing the memory footprint of inference engines.

Conversely, on *the serving side*, systems such as vLLM [30] and SGLang [51] significantly improve inference throughput through techniques like PagedAttention and RadixAttention, enabling efficient continuous batching and prefix sharing. Recent systems including RAGO [24] and RAGCache [26] further optimize configuration and caching strategies for RAG workloads. Nevertheless, these approaches treat retrieved contexts as flat token sequences, remaining agnostic to the underlying graph topology and the varying reasoning difficulty across queries. As a result, they apply uniform computation even when only a small subset of the retrieved evidence is necessary for correct answers.

Although emerging efforts have attempted to bridge this gap, they remain constrained by unidirectional data flows. For instance, DepCache [47] leverages dependency graphs to prune attention computation during inference, and FusionRAG [43] optimizes context splicing for cache reuse. Yet, these methods operate reactively at the serving layer; they cannot proactively regulate the upstream retrieval budget based on real-time system contention, nor can they prioritize critical reasoning paths under strict latency constraints.

This disconnect leads to a brittle *"Phase-Silo" Design* that optimizes intermediate metrics misaligned with end-user experience:

- Retrieval Blindness: Retrieval modules maximize recall without accounting for serving-time costs. Expanding dense neighborhoods can dramatically inflate context length, triggering retrieval floods that immediately violate TTFT SLOs.
- Serving Agnosticism: Serving schedulers optimize generic *Throughput* (total tokens generated per second) but remain blind to *Goodput*—the maximum sustainable request rate under latency SLOs [52]. An aggressive scheduler might prioritize short queries to lower median latency, but it can starve complex multi-hop queries of the compute resources they require, resulting in

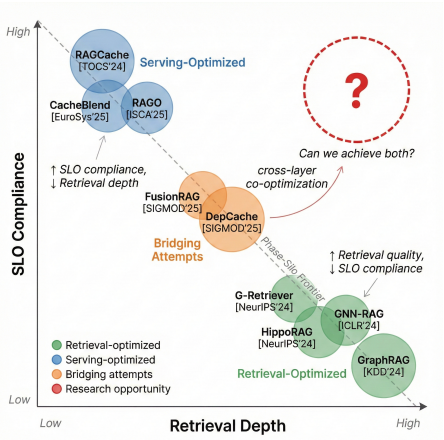


Figure 1: The Quality–Latency Trade-off in RAG Systems. Retrieval-optimized methods [14, 18, 32] (green) achieve high answer quality but violate SLO constraints. Serving-optimized methods [24, 26, 46] (blue) deliver low latency but sacrifice retrieval depth. Bridging attempts [43, 47] (orange) partially close the gap but lack cross-layer feedback. The top-right region remains an open research opportunity.Classification not accurate, need to be discussed.

timeouts or incomplete reasoning.

The core challenge is thus cross-layer coordination: *deciding how much evidence to retrieve and aggregate per query, and when to schedule it, guided by structural signals from the data and system state.*

Our proposal. To bridge these silos, we propose DynaGraph, a holistic serving framework that transforms the linear RAG pipeline into a cooperative closed-loop system. Unlike existing approaches that optimize retrieval recall and serving throughput in isolation, DynaGraph establishes a *bidirectional feedback channel* between the data layer and the compute layer.

DynaGraph *redesigns the retrieval phase to be compute-aware.* Instead of indiscriminately forwarding raw text chunks, the retriever analyzes the topological complexity of the result subgraph (*e.g.,* high-degree "supernodes" or dense cliques) and exposes these signals as *cost priors* to the scheduler. This allows the system to anticipate potential execution bottlenecks before inference begins.

DynaGraph *introduces serving-driven backpressure.* The inference engine continuously monitors real-time SLO slack and hardware contention, and as saturation approaches, it propagates feedback to the aggregation layer to dynamically tighten the evidence budget for incoming queries. This transforms rigid recall–latency conflicts into an adaptive trade-off: context is pruned for latency-tolerant queries, while compute resources are reserved for complex multi-hop reasoning that requires deeper evidence chains.

In summary, this paper makes the following contributions:

- *Topology-to-Cost Mapping (P1):* We propose a structure-aware retrieval interface that exposes graph features (e.g., bridge ratio, entity overlap) not just for reasoning, but as predictive signals for prefill latency, enabling early-stage load awareness.
- *Feedback-Control Aggregation (P2):* We design a feedback-control aggregation controller that regulates the context window size based on real-time serving backpressure. This prevents cache thrashing under heavy load by ensuring that the admitted context volume never exceeds the system's effective processing capacity.
- *QA-Goodput Optimization (P3):* We formalize **QA-Goodput**—a unified metric combining semantic accuracy and SLO compliance. We propose a joint scheduler that prioritizes requests based on their marginal contribution to this objective, achieving a robust balance between correctness and timeliness.
- *End-to-End System Evaluation:* We implement DynaGraph as a **training-free, model-agnostic** middleware on top of latest vLLM/SGLang, requiring no modifications to the serving engine or LLM weights. Experiments on three multi-hop QA benchmarks across various LLM architectures show that DynaGraph achieves **1.9–3.0**× higher QA-Goodput over the best GraphRAG baseline and **2.5**× lower TTFT than static serving. Under end-to-end deployment with live retrieval, DynaGraph maintains E2E TTFT <700 ms across **10**× corpus scaling.

2 PRELIMINARIES AND PROBLEM FORMULATION

In this section, we first introduce the widely used Graph-Enhanced RAG pipeline, and then formalize the problem studied in this paper.

2.1 Graph-Enhanced RAG Pipeline

Knowledge corpus. The knowledge corpus can be modeled as a heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e)$, where $\mathcal{V}$ is the set of nodes representing granular knowledge units (*e.g.,* documents, chunks, and entities), $\mathcal{T}_v$ specifies node types (*e.g., entity, document, chunk*), $\mathcal{E}$ is the set of edges capturing pairwise relationships between nodes, and $\mathcal{T}_e$ defines edge types (*e.g.,* hyperlink, co-occurrence, semantic similarity) [14].

Pipeline. Given the graph-modeled corpus, a Graph-Enhanced RAG pipeline answers a multi-hop query q in three phases:

Phase 1: Retrieval (R). A retriever first identifies a query-specific subgraph $\mathcal{G}_q \subseteq \mathcal{G}$ by combining semantic similarity in embedding space with graph traversal (*e.g.,* community detection [14] or personalized PageRank [18]). This subgraph encodes multi-hop evidence paths for downstream reasoning. Beyond textual content, $\mathcal{G}_q$ also contains topological signals $\mathcal{F}_q$ (*e.g.,* connectivity density, bridge nodes) that reflect reasoning complexity. However, existing approaches typically discard these signals (*e.g.,* [14, 18, 32]).

Phase 2: Aggregation (A). Given the retrieved subgraph $\mathcal{G}_q$, an aggregator selects K evidence chunks X_{ctx} from $\mathcal{G}_q$ (typically via reranking) to construct the context. The resulting prompt is $P = [X_{\text{sys}}, X_{\text{ctx}}, q]$, where X_{sys} provides task-specific instructions. Most systems fix K, imposing a rigid trade-off between context richness and serving cost. Let L_q denote the length of prompt P for query q, which directly determines the prefill workload and thus TTFT. Since K dominates prompt length, we approximate

$$L_q \approx |X_{\text{sys}}| + K \cdot \bar{\ell} + |q|,$$

where $\bar{\ell}$ denotes the average length of the chunks in X_{ctx}.

Phase 3: Generation (G) and Serving. Given prompt P, the LLM generates the answer Y autoregressively, with a prefill stage (*i.e.,* computing the KV cache for P, which dominates TTFT) followed by incremental decoding. Since prefill cost grows super-linearly with prompt length, richer contexts from Phase 2 can significantly increase latency. In practice, queries arrive as a stochastic process with rate $\lambda(t)$ and exhibit diverse reasoning complexity: a single-hop lookup may use K=3 chunks, whereas a multi-hop query may require K=15, leading to order-of-magnitude differences in prefill cost. Under concurrent load, requests compete for GPU memory and compute, requiring the serving engine to coordinate batching, KV caching, and scheduling to meet per-request latency targets. In this paper, we focus on locally served open-source LLMs (*e.g.,* Qwen, Mistral) to enable fine-grained control over these resources.

Example 2: Continuing Example 1, let q' denote the customer-support query: "payment succeeded but order failed; who is affected, and why?" A conventional three-phase pipeline proceeds as follows.

∘ *Phase 1 (Retrieval):* The retriever starts from payment and order entities and traverses $\mathcal{G}$ to extract a query-specific subgraph $\mathcal{G}_{q'}$, linking transaction records, service dependencies, failure traces, inventory/SKU metadata, and recurring incident signatures (*e.g.,* error codes). If $\mathcal{G}_{q'}$ contains a high-degree "Error 503" node (a *supernode*), the query likely lies in a dense fault region and requires additional evidence to disambiguate the root cause.

∘ *Phase 2 (Aggregation):* An aggregator selects a fixed number K (*e.g.,* = 5) of evidence chunks X_{ctx} from $\mathcal{G}_{q'}$ to construct the prompt $P_{q'} = [X_{\text{sys}}, X_{\text{ctx}}, q']$. As causal chains may span multiple services and historical tickets/runbooks, a small K risks missing upstream dependencies, whereas a large K increases token usage and prefill cost.

∘ *Phase 3 (Generation and Serving):* The engine executes $P_{q'}$ via prefill and decoding. With an FCFS (first-come-first-served) scheduler, this time-sensitive query may be delayed behind low-urgency requests, risking TTFT/SLO violations (see failure modes below). □

Limitations. Modern serving engines such as vLLM [30] and SGLang [51] improve throughput and reduce redundant computation through KV-cache management (*e.g.,* PagedAttention). However, they remain inadequate for GraphRAG workloads, where query difficulty and context size vary widely, as discussed below (see Section 3 for empirical evidence).

Topology-Blind Scheduling. Existing schedulers (*e.g.,* FCFS, continuous batching) treat requests as homogeneous units, ignoring the substantial variance in prefill cost induced by graph topology. For example, a multi-hop query traversing a dense subgraph (*e.g.,* around a supernode) may require significantly more context and prefill time than a single-hop lookup, yet both receive identical priority. This mismatch leads to *head-of-line (HOL) blocking* [27], where complex queries delay simpler, latency-sensitive ones (or vice versa).

Static Context Budget. The context budget K is typically fixed: the aggregator selects K chunks regardless of runtime load. Under high contention, a large K inflates prompt length and prefill latency, exacerbating queueing delays; under light load, a small K underutilizes available capacity and may degrade answer quality. Existing systems rarely adapt K to current serving conditions.

No Cross-Layer Feedback. Retrieval, aggregation, and serving form an open-loop pipeline. The serving layer cannot propagate congestion signals upstream to adjust K, while the retriever/aggregator cannot provide cost estimates that inform scheduling decisions. Without cross-layer feedback, the system cannot proactively adapt to load fluctuations or heterogeneous query complexity.

Failure modes. With the limitations above, existing approaches exhibit two common failure modes.

Fast but wrong (mode (a)). When context is insufficient (*i.e., K* too small), the LLM lacks critical evidence and may fabricate answers [21]. Worse, for reasoning-augmented models, missing grounding can also induce prolonged exploratory reasoning, increasing decode cost without improving accuracy [9].

Accurate but late (mode (b)). With excessive context (*i.e., K* too large), prefill dominates latency: the answer is usually correct, but arrives after the user has moved on.

Both failures make the response useless: mode (a) tends to arise for complex queries when K is small, whereas mode (b) can occur even for simple queries when K is large.

Example 3: Continuing Example 2 with an SLO threshold τ=2 s, consider q' under two static configurations:

∘ With K=5, the retriever retains only payment-callback logs and misses the upstream inventory-lock root cause; the LLM thus hallucinates a "gateway timeout" (TTFT ∼180 ms; failure mode (a)).

∘ With K=20, all relevant evidence is included and the root cause is correctly identified, but the ∼4,000-token prompt raises TTFT to ∼2,800 ms, exceeding the SLO (failure mode (b)). □

2.2 Metric and Problem Formulation

In this subsection, we introduce a new evaluation metric for graph-enhanced RAG, upon which we formalize our problem.

Metric. We first review the widely used Goodput metric and its limitations, and then introduce a new metric for graph-enhanced RAG that jointly accounts for answer accuracy and a hard deadline.

Goodput [52]. It quantifies the maximum sustainable request arrival rate subject to a latency Service Level Objective (SLO), specified as a per-request deadline τ_q on TTFT (time-to-first-token):

$$\text{Goodput} = \max\ \lambda \quad \text{s.t.} \quad \Pr\big(\text{TTFT} \le \tau_q\big) \ge P \tag{1}$$

where λ is the request arrival rate and $P \in (0, 1]$ is the target SLO-attainment probability.

As shown in Eq. (1), Goodput accounts only for timeliness; it treats a correct on-time answer and an on-time hallucination equally, leaving failure mode (a) unaddressed.

QA-Goodput. To capture the failure modes in Section 2.1, we introduce *QA-Goodput* $\mathcal{J}_\lambda$, which (a) assigns zero credit to any response that misses its deadline, regardless of correctness, and (b) penalizes low-quality answers. Formally, for a query set $\mathcal{S} = \{q_i\}_{i=1}^{N}$ at

arrival rate λ, where each query q_i has a TTFT deadline τ_{q_i}, we define:

$$\mathcal{J}_\lambda = \frac{1}{N}\sum_{i=1}^{N} \underbrace{\text{Score}(q_i)}_{\text{quality}} \cdot \underbrace{\mathbb{I}(\text{TTFT}_i \le \tau_{q_i})}_{\text{timeliness}} \tag{2}$$

where $\text{Score}(q_i) \in [0,1]$ measures answer quality (*e.g.*, token-level F1 [37]), and $\mathbb{I}(\cdot)$ is an indicator function. Intuitively, a higher $\mathcal{J}_\lambda$ indicates that more queries in Section are answered accurately and within their deadlines, thereby mitigating both failure modes.

Remark. (1) Goodput considers only timeliness, counting any on-time response as successful regardless of correctness. In contrast, QA-Goodput couples quality with a hard deadline, assigning zero credit to late responses and penalizing low-quality ones. (2) QA-Goodput captures settings requiring *both* high answer quality and strict deadline compliance, complementing Goodput.

Problem. We formalize the problem studied in this paper as follows.

- *Input:* a document corpus $\mathcal{D}$ (from which a heterogeneous graph $\mathcal{G}$ is constructed offline, a query batch $\mathcal{S} = \{q_i\}_{i=1}^n$ with arrival rate λ, where each query q_i has a TTFT deadline τ_{q_i} and an admissible retrieval/aggregation budget $K_i \in [K_{\min}, K_{\max}]$.
- *Output:* a serving order π of queries in $\mathcal{S}$ and per-query budgets $\mathcal{K} = \{K_1, \ldots, K_n\}$.
- *Objective:* maximize QA-Goodput under query budget constraints:

$$\max_{\mathrm{K},\pi} \sum_{i=1}^{n} Q(K_i, q_i) \cdot \mathbb{I}(\text{TTFT}_i \le \tau_{q_i}) \quad \text{s.t. } K_i \in [K_{\min}, K_{\max}],\ \forall i \tag{3}$$

where $Q(K_i, q_i)$ denotes the answer score (cf. Eq. (2)) when serving q_i with K_i evidence chunks.

Remark. (1) The problem requires jointly optimizing two coupled decisions: the serving order π (scheduling) and the per-query budgets $\mathcal{K}$ (budget allocation). (2) The interval $[K_{\min},, K_{\max}]$ defines the feasible budget range supported by the system and can be set based on prior knowledge of the workload. (3) Since scheduling and budget allocation jointly influence both quality and timeliness, the problem is more challenging than standard deadline scheduling with fixed processing times (see below).

Hardness. The restricted decision version of the studied problem in Eq. (3) is to decide, given a query set $\mathcal{S}$, arrival rate λ, per-query deadlines τ_{q_i} for each $q_i \in \mathcal{S}$, a fixed budget vector K, and a threshold η, whether there exists a serving order π on a non-preemptive single machine such that $\sum_{i=1}^n Q(K_i, q_i) \cdot \mathbb{I}(\text{TTFT}_i \le \tau_{q_i}) \ge \eta$. We show that even this restricted version is intractable.

Theorem 1: *The above restricted decision problem is NP-hard.* □

Proof. We establish NP-hardness by a polynomial-time reduction from Partition [28]. Details are deferred to Appendix A.1.

Computational Hardness. Even with K fixed, the scheduling sub-problem is intractable:

Theorem 2: *[NP-Hardness] Given n queries with fixed processing times p_i, deadlines τ_i, and values v_i, finding a non-preemptive single-machine schedule that maximizes $\sum_{i:\, C_i \le \tau_i} v_i$ is NP-hard.* □

Proof. By reduction from Partition [28]; see Appendix A.1. Since ($\mathcal{P}$) additionally optimizes over K, it is at least as hard.

Decomposition. ($\mathcal{P}$) embodies three coupled trade-off axes: (1) *Quality vs. Cost*: larger K_i improves Q but inflates p_i; (2) *Latency vs. Throughput*: prioritizing urgent requests improves their TTFT at the expense of others; (3) *Local Quality vs. Global Goodput*: under congestion, compressing K_i sacrifices individual quality for system-wide SLO compliance. DynaGraph decomposes ($\mathcal{P}$) via a closed-loop architecture: Phase 2 solves the resource-allocation sub-problem (setting K using both dimensions of the cost model), Phase 3 solves the scheduling sub-problem (choosing π), and a backpressure signal S_{sys} communicates Phase 3's congestion state back to Phase 2 (Section 4.4–Section 4.6).

3 MOTIVATION AND ANALYSIS

In this section, we describe the test setup for the evaluated phase-silo pipelines (Section 3.1), present empirical evidence of their limitations (Sections **??**–3.4), and summarize the key observations motivating our methodology (Section 3.5).

3.1 Setup

Dataset. We use 2WikiMultihopQA [20], a two-hop QA benchmark with four reasoning types: (a) *compositional* (n=413, chaining two facts, *e.g.*, "capital of X's birthplace"), (b) *comparison* (n=244, comparing attributes of two entities), (c) *bridge_comparison* (n=235, requiring cross-document bridging and comparison), and (d) *inference* (n=108, deriving implicit conclusions from stated facts). We treat *bridge_comparison* as "Hard" due to its need for cross-document bridging, and the remaining types as "Easy", as they are less sensitive to the context budget K. We adopt this dataset for our motivation study because its annotated reasoning types enable controlled stratification by difficulty.

Workload. We model the serving workload along two orthogonal dimensions: *arrival pattern* and *query popularity*. Requests arrive according to a Poisson process with rate λ, consistent with standard RAG-serving evaluations [26, 38]. To capture the long-tailed reuse common in knowledge-intensive QA, we draw queries from a Zipfian popularity distribution (α=1.0) [6]; this is motivated by (a) power-law degree distributions in knowledge graphs [3] and (b) query logs from knowledge-graph endpoints showing heavily skewed entity access [5]. It increases prompt-length variance and exacerbates HOL blocking, making scheduling more challenging than under uniform workloads; it also helps more clearly differentiate processing configurations (see below) under realistic conditions.

Configurations. We evaluate four configurations that a graph-enhanced RAG pipeline may adopt:

- *Static (C1):* Fixed context budget K=20 with FCFS scheduling, reflecting the common phase-silo baseline.
- *Schedule-Only (C2):* Fixed K=20 with SLO-aware priority scheduling via expert-defined, workload-specific policies, isolating the impact of scheduling.
- *Adaptive-K (C3):* Difficulty-aware adaptive K with FCFS sched-

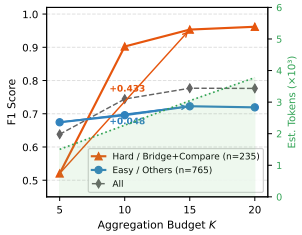


Figure 2: The performance of static configuration under different context budgets K. Green shading shows prefill tokens (right axis), which increases regardless of query difficulty.

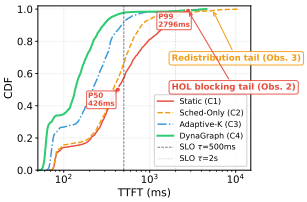


Figure 3: TTFT distribution (Obs. 2–3). Static (C1) exhibits a heavy tail from HOL blocking. Schedule-Only (C2) improves the median but extends the tail further (P99: 6,484 ms vs. 2,796 ms). Joint (C4) shifts the entire distribution left.

uling, where query difficulty is provided as expert input, isolating the impact of budget adaptation.

- *Joint (C4):* Difficulty-aware adaptive K with SLO-aware scheduling, both guided by expert-defined policies as above.

These configurations isolate *budget adaptation* (C1→C3) and *scheduling* (C1→C2), with C4 evaluating their joint effect.

Infrastructure. All experiments run on a single NVIDIA RTX 4090 (24 GB). We use Qwen3-4B with Zipfian arrivals (λ=30 req/s; 1000 queries × 3000 requests; concurrency 64).

Quality Metric. Answer quality is measured by token-level F1 [37], corresponding to Score(q_i) in the QA-Goodput definition (Eq. 2).

3.2 Observations

Under the test setup in Section 3.1, we observe the following.

(1) The context size K matters. We evaluated the static configuration (C1) across four reasoning types by varying the aggregation budget K from 5 to 20 in Figure 2, reporting F_1 (left axis) and prefill-token consumption (right axis). Prefill tokens increase monotonically with K, but the quality–cost trade-off differs sharply across query difficulty: hard queries benefit substantially from larger K, while easy queries show only marginal gains. For example, increasing K from 5 to 15 raises hard-query F_1 from 0.52 to 0.95, *e.g.*, 9× the gain observed for easy queries, since small K often misses key evidence for multi-step reasoning. However, larger K does not always improve accuracy: increasing K from 15 to 20 reduces F_1 for easy queries (and the overall average), indicating that overly large contexts can introduce distracting noise.

Overall, fixed-K aggregation wastes budget on easy queries while under-serving hard ones, making appropriate selection of K essential: too small yields insufficient evidence, while too large increases cost and may degrade quality.

(2) Query difficult can be used for setting K but leads to sub-optimal results. (a) show providing query difficult as input may still violates the deadline; (b) state that existing approches don't know the query difficult in advance.

(3) Scheduling alone cannot address the QA-goodput problem. currently this part is ok, but better providing an individual figure for illustrating this issue.

(4) Joint optimization is promising but needs to be implemented automatically. (a) Show that the joint configuration (C4) is the best, *e.g.*, highest F_1 and lowerest prefill cost compared to other three configurations. (b) mention that C4 is set by experts manually, which is promising but realistic (thus should be implemented automatically, naturally motivating Section 4).

Table 1: Motivation: configuration comparison. Under strict SLO (τ=200 ms), Joint (C4) achieves 3.8× higher $\mathcal{J}_\lambda$ than Static (C1), while Sched-Only (C2) worsens P99 by 2.3×. (Qwen3-4B; 2WikiMultiHopQA; Zipf λ=30 req/s; 1000q×3000 requests.)

Config	Quality F1	Quality avg K	Latency (↓) P50	Latency (↓) P99	τ=200 ms (↑) SLO	τ=200 ms (↑) $\mathcal{J}_\lambda$	τ=500 ms (↑) SLO	τ=500 ms (↑) $\mathcal{J}_\lambda$
Static (C1)	0.758	20.0	426 ms	2796 ms	18.7%	16.3%	57.2%	47.2%
Sched-Only (C2)	**0.762**	20.0	361 ms	6484 ms	21.6%	18.8%	66.0%	54.9%
Adaptive-K (C3)	0.750	14.7	218 ms	**2224 ms**	45.9%	38.3%	91.9%	74.0%
Joint (C4)	0.741	**10.1**	**136 ms**	2288 ms	**74.4%**	**61.4%**	**97.8%**	**78.4%**

Table 2: Motivation: configuration comparison. (Qwen3-4B; 2WikiMultiHopQA; Zipf λ=30 req/s; 1000q×3000 requests.)

Config	Quality F1	Quality avg K	Latency (↓) P50	Latency (↓) P99	τ=200 ms (↑) SLO	τ=200 ms (↑) $\mathcal{J}_\lambda$	τ=500 ms (↑) SLO	τ=500 ms (↑) $\mathcal{J}_\lambda$
Static (C1)	0.758	20.0	426 ms	2796 ms	18.7%	16.3%	57.2%	47.2%
Sched-Only (C2)	**0.762**	20.0	361 ms	6484 ms	21.6%	18.8%	66.0%	54.9%
Adaptive-K (C3)	0.750	14.7	218 ms	**2224 ms**	45.9%	38.3%	91.9%	74.0%
Joint (C4)	0.741	**12.5**	**136 ms**	2288 ms	**74.4%**	**61.4%**	**97.8%**	**78.4%**

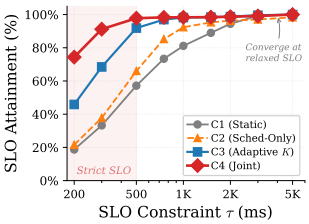


Figure 4: SLO frontier. Joint (C4) dominates at strict deadlines ($\tau \leq 500$ ms); all configurations converge at $\tau \geq 2$ s.

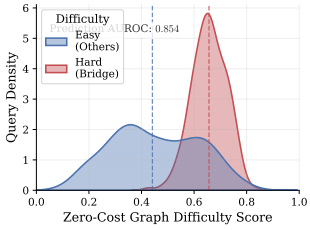


Figure 5: Zero-cost difficulty prediction (Obs. 4). Distribution of graph-derived difficulty score for Easy (n=765) and Hard (n=235) queries. Combined AUROC = 0.854.

3.3 Scheduling Alone Cannot Resolve Congestion

Obs. 2: Scheduling alone cannot resolve head-of-line blocking issue. GraphRAG queries exhibit high variance in prompt length and difficulty due to supernode retrieval. Under Static (C1), per-request token count ranges from ~1,852 to ~7,466. This variance causes head-of-line (HOL) blocking: P99 TTFT reaches 2,796 ms, with ~43% of requests missing the τ=500 ms SLO (Figure 3, red curve). The straightforward response is to apply SLO-aware priority scheduling, yet this alone is insufficient. Sched-Only (C2) improves the median but *extends* the tail to 6,484 ms (2.3× worse than C1), because all requests still carry the full K=20 context unchanged. Reordering cannot reduce the aggregate prefill workload; it merely redistributes latency from high-priority to low-priority requests (Figure 3, orange curve). This observation establishes that cost reduction via adaptive K is a necessary condition for effective scheduling.

3.4 Zero-Cost Graph Features Identify

Complex Queries

Obs. 3: Query difficulty is heterogeneous but predictable in structure. The quality curves in Figure 2 reveal that the optimal budget K varies sharply across query types. Hard queries require $K \geq 15$ to recover bridging evidence (F1 increases from 0.52 to 0.95), while Easy queries saturate at $K \approx 5$–8 (F1 plateau at 0.68–0.72). A difficulty-aware oracle that assigns per-query K based on ground-truth labels could reduce avg K by ~50% with only −0.017 F1 loss (Table 1, C4 vs C1). However, ground-truth difficulty labels are unavailable at serving time. This motivates a zero-cost estimator derived from signals already present in the retrieval pipeline, which we examine in Obs. 4.

Obs. 4: Zero-cost topological features predict query difficulty. Obs. 3 shows that per-query difficulty estimation is essential but are unavailable at serving time. We observe that topological features extracted as byproducts of Phase 1 retrieval provide a reliable proxy without additional inference. Specifically, the fraction of bridging documents (r_{bridge}) and the entity overlap density across retrieved chunks (ρ_{overlap}, defined in Section **??**) each achieve AUROC>0.84 for distinguishing Hard from Easy queries. A linear combination of these features reaches AUROC = **0.854** (Figure 5), because multi-hop queries produce structurally distinct subgraphs (more bridging nodes, denser cross-chunk overlap) regardless of their surface-level phrasing. These scores initialize both budget allocation (Phase 2) and scheduling priority (Phase 3), with runtime feedback for correction under load.

3.5 Summary and Design Implications

These observations motivate three design principles:

(1) **Adaptive aggregation** is the primary lever for the quality–cost trade-off (Obs. 1). Allocating budget per query based on difficulty reduces avg K from 20 to 10.1 with only −0.017 F1 loss, yielding **3.8**× higher $\mathcal{J}_\lambda$@200ms than Static (Table 1). The latency improvement is substantial: P50 drops from 426 ms to 136 ms (−68%).

(2) **SLO-aware scheduling with backpressure** is essential under strict SLOs (Obs. 2). Scheduling alone redistributes rather than reduces load: Sched-Only (C2) improves the median but worsens P99 to 6,484 ms. Combining scheduling with adaptive K and backpressure feedback enables +28.5 pp SLO@200ms over Adaptive-K alone (C3: 45.9% → C4: 74.4%), as backpressure triggers budget compression under congestion, preventing the tail inflation that scheduling alone cannot resolve.

(3) **Graph topological features** provide accurate, zero-cost initialization signals (Obs. 3–4). With AUROC = 0.854, these features bootstrap both the difficulty scorer and scheduling priority without additional inference overhead.

Figure 4 summarizes the full SLO spectrum: Joint (C4) dominates at strict deadlines ($\tau \leq 500$ ms), while C1, C3, and C4 converge at relaxed thresholds ($\tau \geq 2$ s).

While Joint (C4) demonstrates that combining adaptive budgets, priority scheduling, and backpressure yields substantial gains, the configuration tested here relies on hand-tuned parameters for each module. Automating these decisions, specifically (a) estimating per-query difficulty from graph topology, (b) translating difficulty into a budget under varying load, and (c) closing the feedback loop between scheduler and aggregator, is the challenge addressed by DynaGraph (Section 4).

4 METHODOLOGY

In this section, we first provide an overview of our proposed system (Section 4.1) and then detail its main components in Sections 4.2–4.5.

4.1 System Overview

Overview. We present DynaGraph, a holistic serving framework for maximizing QA-Goodput (Section 2.2) under stochastic arrival loads. As shown in Figure 6, unlike existing pipelines that execute retrieval and serving in isolation (*i.e.,* feed-forward), DynaGraph establishes a *bidirectional feedback loop* between data complexity (Graph) and system capacity (Serving). Moreover, it decomposes the joint optimization problem (Eq. 3) into three coordinated phases that together address the pain points identified in Section 3.2.

The key ideas of the three phases are as follows.

Phase 1: Retrieval & Signal Extraction (Sensor). This phase takes a query $q \in \mathcal{S}$ and the global graph $\mathcal{G}$ as input, and produces (a) a top-N candidate subgraph $\mathcal{G}_q \subseteq \mathcal{G}$ and (b) a topological feature vector $\mathcal{F}_q$ that captures query-relevant structural statistics. Concretely, DynaGraph queries an offline heterogeneous graph index (document/chunk/entity layers) to retrieve $\mathcal{G}_q$ and compute $\mathcal{F}_q$ (*e.g.,* bridge ratio, overlap density) to estimate query complexity (cf. Obs. 3–4, Section 3.4). These lightweight signals guide difficulty-aware context budgeting K_q (Phase 2) for each query and the scheduling order π (Phase 3) across the query set $\mathcal{S}$, prior to expensive LLM inference.

Phase 2: Adaptive Aggregation (Controller). This phase takes $\mathcal{G}_q$, $\mathcal{F}_q$, and real-time system backpressure S_{sys} (*e.g.,* queue utilization from the serving engine) as input, and outputs an aggregation budget K_q and an optimized prompt P_q with dynamic length L_q. To address the context-size challenge in Obs. 1 (Section 3.2), it balances query difficulty against instantaneous load by allocating more context to hard queries while throttling easy ones under congestion. That is, it selects K_i for each query q_i to maximize expected quality $Q(K_i, q_i)$, thereby improving overall QA-Goodput across $\mathcal{S}$.

Phase 3: SLO-Aware Scheduling (Actuator). Phase 3 takes as input the prompt stream $\{P_q\}$ for queries in $\mathcal{S}$, per-query SLO constraints $\{\tau_{q_i}\}$ for $q_i \in \mathcal{S}$, and cost/urgency estimates $(\hat{C}_i, \hat{D}_i)$ derived from $\mathcal{F}_q$, and outputs the execution order π and corresponding inference results Y. To mitigate head-of-line (HOL) blocking (Obs. 2) and address the limitations of cost-agnostic scheduling, it replaces FCFS with an SLO-aware priority queue that ranks requests by *SLO urgency* and *estimated cost* [17], and selects π that maximizes the QA-goodput (*e.g.,* $\sum_{i:\,\text{TTFT}_i \leq \tau_{q_i}} Q(K_i, q_i)$) given the budget vector K from Phase 2.

Note. (1) Phase 3 continuously monitors serving-side congestion (*e.g.,* queue depth and token-budget utilization) and feeds a *congestion signal* S_{sys} back to Phase 2. Under overload, S_{sys} triggers more aggressive budget reduction, closing the feedback loop. Thus, con-

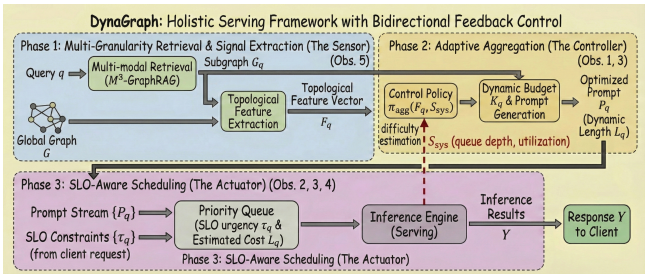


Figure 6: DynaGraph Architecture. The framework operates as a closed-loop control system. Phase 1 (Sensor) extracts topological priors from an offline-built heterogeneous graph index; Phase 2 (Controller) dynamically budgets context K_q based on difficulty and backpressure (Obs. 1, 3–4); Phase 3 (Actuator) schedules requests by urgency and cost (Obs. 2), while feeding congestion signals back to Phase 2.

Table 3: Summary of Key Notations.

Symbol	Description
	Metric & Problem (Section 2.2)
$\mathcal{S} = \{q_i\}_{i=1}^{n}$	Query batch (n queries)
λ	Request arrival rate
τ_{q_i}	TTFT deadline for query q_i
$\text{Score}(q_i)$	Answer quality $\in [0,1]$ (e.g., token-level F1)
$\mathcal{J}_\lambda$	QA-Goodput (Eq. 2)
$Q(K_i, q_i)$	Expected answer quality $\mathbb{E}[\text{Score}(q_i) \mid K_i]$
$\mathrm{K} = (K_1, \ldots, K_n)$	Per-query budget vector (decision variable)
π	Serving order / scheduling permutation (decision variable)
	Graph & Retrieval (Phase 1)
$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e)$	Global heterogeneous knowledge graph
q, $\mathcal{G}_q$	User query and its induced candidate subgraph
$\mathcal{D}$	Raw document corpus (offline input)
$\mathcal{F}_q$	Topological feature vector of $\mathcal{G}_q$ (Sensor signal)
r_{bridge}	Bridge ratio: fraction of chunks connecting ≥ 2 entity clusters
ρ_{overlap}	Entity overlap density: fraction of chunk pairs sharing ≥ 1 entity
n_{entity}	Number of unique entities in $\mathcal{G}_q$
	Adaptive Aggregation (Phase 2)
$S_{\text{diff}}(q)$	Normalized difficulty score $\in [0,1]$
S_{sys}	Binary backpressure gate: $\mathbb{I}(u(t) > \theta_{\text{bp}})$
θ_{bp}	Backpressure activation threshold
K_q	Final aggregation budget for query q
$K_{\min}$, $K_{\max}$, K_{eff}	Lower / upper / backpressure-adjusted budget bounds
$L_i(K_i)$	Prompt length (tokens) given budget K_i
$p_i = L_i(K_i)/\mu$	Estimated prefill time for query q_i
μ	Serving engine processing rate (tokens / ms)
$u(t)$	Instantaneous queue utilization $\in [0,1]$
	Scheduling (Phase 3)
t_{arr}, $t_{\text{wait}}(q)$	Arrival time and elapsed wait time

straint violations in scheduling are propagated to resource allocation, enabling K and π to co-adapt rather than be optimized independently. (2) Overall, the Sensor provides priors, the Controller allocates resources, and the Actuator executes while reporting congestion; together, they form a feedback-control system [19] that adapts to transient load spikes without manual tuning.

4.2 Retrieval & Signal Extraction (The Sensor)

Overview. Given a user query q in a query set $\mathcal{S}$ and a document corpus $\mathcal{D}$, this phase builds an offline heterogeneous graph $\mathcal{G}$ (if not already available) with three node types: Document, Chunk, and Entity. It then induces a query subgraph $\mathcal{G}_q$ via two-hop structure-semantic retrieval: Hop-1 performs semantic anchoring to obtain initial nodes, and Hop-2 expands from the induced seeds through graph structure to surface bridging evidence. All retrieved candidates are jointly re-ranked into a unified list C_q. Finally, it profiles $\mathcal{G}_q$ online to extract a lightweight topological feature vector $\mathcal{F}_q$ (*e.g.*, bridge-related connectivity statistics), providing high-recall structural priors for downstream budgeting and scheduling.

Heterogeneous graph construction. Given corpus $\mathcal{D}$, we construct (if not already available) an offline heterogeneous graph index $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e)$ (defined in Section 2.1) in three stages. First, each document $d \in \mathcal{D}$ is segmented into semantic chunks that preserve sentence boundaries [22]. Second, an offline LLM extracts entities and relational triples from each chunk via schema-constrained pipelines [10, 36, 48], producing canonicalized entity nodes grounded to a reference ontology (*e.g.*, Wikidata [42]). Third, we construct the heterogeneous graph by linking documents, chunks, and entities, forming inclusion edges (Document→Chunk→Entity) and relational edges (Entity↔Entity).

This hierarchical structure can expose two key structural phenomena for GraphRAG serving: *supernodes* (entities referenced by many chunks, inducing prompt-length variance; Obs. **X**) and *bridges* (entities connecting otherwise disjoint evidence regions, often requiring larger context budgets; Obs. **X**).

Structure-Semantic Collaborative Retrieval. Given query q and constructed $\mathcal{G}$, it induces a candidate subgraph $\mathcal{G}_q \subseteq \mathcal{G}$ and a ranked candidate corpus C_q via a two-hop coarse-to-fine strategy (seed-expand) [18, 25, 35]. It proceeds in three steps: (1) *Hop-1 (semantic anchoring)* retrieves the top-K_1 chunks by bi-encoder cosine similarity to form C_q^{chunk}, collects their parent documents as C_q^{doc}, and traverses inclusion edges to gather mentioned entities as the seed set; (2) *Hop-2 (structure-guided expansion)* traverses up to two relational hops from the seeds to surface bridging evidence, then maps newly discovered entities back to their source chunks via inclusion edges to form C_q^{graph}, with Hop-1 and Hop-2 jointly constituting $\mathcal{G}_q$; and (3) *unified scoring* jointly re-scores $C_q = C_q^{\text{chunk}} \cup C_q^{\text{doc}} \cup C_q^{\text{graph}}$ using a cross-encoder reranker to produce $s_{\text{rerank}}(d, q)$ and a single ranked list, while retaining bi-encoder embeddings e_d for coverage-aware assembly in Phase 2 (Section 4.4.4).

Topological signal extraction. Given the query-induced subgraph $\mathcal{G}_q$, DynaGraph applies a lightweight profiling step to map $\mathcal{G}_q$ to a feature vector $\mathcal{F}_q \in \mathbb{R}^d$, rather than directly passing it to generation. These features serve as *zero-cost structural priors* for downstream control (Phases 2–3): they are inexpensive to compute and informative for initialization, though not used as standalone predictors. Let $\mathcal{E}(c)$ denote the entity set of a chunk c in $\mathcal{G}_q$, and let $C_q^{\mathcal{G}}$ denote the set of chunk nodes in $\mathcal{G}_q$. We introduce three key signals in $\mathcal{F}_q$ that will be used in Phase 2 (Eq. 5):

- *Bridge ratio* r_{bridge}: the fraction of chunks $c \in C_q^{\mathcal{G}}$ that share entities with at least two other chunks, i.e., act as connectors across otherwise disjoint evidence regions. A higher r_{bridge} indicates stronger cross-chunk connectivity and thus a more multi-hop query that typically requires a larger aggregation budget

K (Obs. 1); empirically it provides the strongest single-feature separation between easy and hard queries.

- *Entity overlap density* ρ_{overlap}: the fraction of chunk pairs in $\mathcal{G}_q$ that share at least one entity, $\rho_{\text{overlap}} = \left|\{(c_i, c_j) : i<j,\ \mathcal{E}(c_i) \cap \mathcal{E}(c_j) \neq \emptyset\}\right| / \binom{|C_q^{\mathcal{G}}|}{2}$. This captures dense local connectivity, complementary to the global bridging structure measured by r_{bridge}, and yields the largest marginal gain when combined with r_{bridge}.
- *Entity count* n_{entity}: the number of unique entities covered by $\mathcal{G}_q$, $n_{\text{entity}} = \left|\mathcal{V}_{\text{entity}} \cap \mathcal{G}_q\right|$. Larger n_{entity} reflects broader evidence requirements and tends to correlate with longer reasoning chains, making the query harder.

Example 4: For the e-commerce support query q' ("payment succeeded but order failed …"), Phase 1 induces a subgraph $\mathcal{G}_{q'}$ that links evidence around *order*, *payment*, *inventory*, and *ticket*. It then extracts the corresponding feature vector $\mathcal{F}_{q'}$ (*e.g.*, flagging the "Error 503" entity as a supernode). The Controller (Phase 2) and Actuator (Phase 3) consume these signals to drive difficulty-aware budgeting and scheduling before the expensive prefill stage. □

Note. Extracting $\mathcal{F}_q$ involves only simple combinatorial operations over a small subgraph (typically < 500 nodes) and adds negligible overhead (typically < 5 ms in our implementation). This provides Phase 2 with an early signal that helps disentangle quality-driven budget needs from system-driven cost control.

4.3 Phase 2: Adaptive Aggregation

4.4 Phase 2: Adaptive Aggregation

Phase 1 delivers the subgraph $\mathcal{G}_q$, topological signals $\mathcal{F}_q$, and a ranked candidate corpus $C_q = C_q^{\text{chunk}} \cup C_q^{\text{doc}} \cup C_q^{\text{graph}}$ (Section **??**). Phase 2 performs *adaptive context compression*: it selects $K_q \in [K_{\min}, K_{\max}]$ evidence units from C_q, compressing the full retrieval output into a prompt that preserves answer-critical evidence while shedding redundancy. The compression ratio is query-specific (governed by a difficulty estimate), load-responsive (governed by a backpressure ceiling), and *training-free by design*: all signals and embeddings are reused from Phase 1 with no additional model inference, ensuring that Phase 2 itself adds negligible latency. To formalize the cost implications of this compression, we first establish a per-query cost model.

Cost Model. In autoregressive LLM inference, each request incurs two sequential costs with distinct computational profiles [1, 52]. For query q_i served with budget K_i, the total GPU occupancy is:

$$c_i(K_i, q_i) = \underbrace{L_i(K_i) \,/\, \mu_{\text{p}}}_{\text{prefill (compute-bound)}} + \underbrace{\hat{D}_i(q_i) \,/\, \mu_{\text{d}}}_{\text{decode (memory-bound)}} \tag{4}$$

where $L_i(K_i) = |X_{\text{sys}}| + K_i \cdot \bar{\ell} + |q_i|$ is the prompt length (tokens), μ_{p} is the prefill throughput (tokens/ms), $\hat{D}_i(q_i)$ is the expected output length, and μ_{d} is the decode throughput (tokens/ms). The prefill term is *controllable*: K_i directly determines L_i, and hence TTFT_i [24, 38]. The decode term is *observable but not controllable*: $\hat{D}_i$ depends on the intrinsic reasoning complexity of q_i and cannot be reduced by budget compression. The prefill term directly determines system congestion, since pending prefill workload dictates when each request can begin generation; ongoing decode contributes indirectly through GPU compute contention and KV-cache residency. This asymmetry drives a two-lever design: Phase 2 controls the prefill cost via adaptive K_i, while Phase 3 monitors $u(t)$ to capture the combined effect of both phases on system congestion.

4.4.1 The Budget Allocation Problem. The central challenge is that a fixed budget K is structurally suboptimal. Obs. 1 shows that the oracle budget K^*—the minimum K preserving answer quality—is heavily skewed: the vast majority of queries need only a small context, while a minority of complex bridge-comparison queries require substantially more evidence. A static pipeline provisions $K_{\max}$ for every query, wasting prefill tokens on simple queries and inflating latency uniformly. Conversely, Obs. 3 shows that scheduling alone cannot compensate: reordering uniformly heavy requests merely redistributes latency without reducing aggregate load.

Example 5: Continuing Example 3, suppose query q' ("payment succeeded but order failed") arrives concurrently with a simple status-check query q'' ("what is order #1234's shipping date?"). A static pipeline allocates K=20 to both. Yet q'' is answerable from a single order-record chunk (K^*=2), while q' requires the inventory-lock event, service-dependency traces, and historical tickets ($K^* \geq 12$). The excess context for q'' wastes prefill tokens and inflates its TTFT, while q', queued behind q'' under FCFS, risks missing its SLO. Phase 2 must assign $K_{q'} \gg K_{q''}$ automatically, using only signals available *before* LLM inference. □

Phase 2 addresses this by acting as a **controller** that dynamically sets K_q through three coordinated mechanisms: **(i)** a *difficulty estimator* $S_{\text{diff}}(q)$ that determines how aggressively to compress each query's context (Section 4.4.2); **(ii)** a *load-dependent ceiling* $K_{\text{eff}}(t)$ that bounds the maximum budget under congestion (Section 4.4.3); and **(iii)** a *coverage-aware assembler* that selects *which* documents to retain when the budget is compressed (Section 4.4.4). Neither signal alone is sufficient; their interaction creates the feedback loop that simultaneously optimizes quality and latency.

4.4.2 Topological Difficulty Estimation. We need a scoring function $S_{\text{diff}} : Q \to [0, 1]$ that separates queries like q' from q'' (Example 5) *without* any LLM inference, to minimize the latency overhead of Phase 2 itself. Obs. 4 establishes that topological features extracted during Phase 1 reliably distinguish hard from easy queries (AUROC = 0.854), and crucially, these signals are zero-cost byproducts of the retrieval stage.

We exploit three structural properties of the retrieved subgraph, each capturing a distinct dimension of multi-hop complexity:

- **Bridge ratio** r_{bridge}: the fraction of documents connecting otherwise disjoint entity clusters. Bridge-comparison queries like q' produce subgraphs with high bridge ratios because the answer path necessarily spans multiple entity clusters (payment → inventory-lock → order failure). This feature achieves the strongest individual separation between hard and easy queries.
- **Overlap density** ρ_{overlap}: the fraction of document pairs

sharing at least one entity in $\mathcal{G}_q$. This captures dense local connectivity—a complementary signal to the global bridging structure of r_{bridge}—and yields the largest marginal gain when combined with the bridge ratio.

- **Entity count** n_{entity}: the number of unique entities in $\mathcal{G}_q$, reflecting evidence breadth. A fourth candidate feature (*e.g.,* maximum entity degree) provides no further improvement, confirming three features as the effective ceiling.

Let $\tilde{\mathrm{f}}_q$ denote the min-max normalized feature vector. The difficulty score is a linear combination:

$$S_{\text{diff}}(q) = \mathrm{w}^\top \tilde{\mathrm{f}}_q, \tag{5}$$

where $\mathrm{w} \in \mathbb{R}^3$ is calibrated via grid search on held-out oracle data (detailed in Section 5.6). For query q' in Example 5 (high bridge ratio across payment/inventory clusters, moderate overlap, many entities), $S_{\text{diff}} \approx 1$; for the simple status-check q'' (no bridging, low overlap, few entities), $S_{\text{diff}} \approx 0$.

Remark (Design Rationale). S_{diff} is deliberately a *coarse continuous classifier* (hard vs. easy), not a precise regressor of K^*. This design choice reflects a fundamental distinction: the features that best *classify* difficulty (r_{bridge}, ρ_{overlap}) differ from those that best *predict* the oracle budget, because binary classification requires only separability, whereas budget prediction demands precise localization of gold evidence in the ranking [50]. Learned context selectors [13, 50] and complexity classifiers [23] pursue this harder objective via per-dataset supervised training, while LLM-based profilers [38] invoke a separate LLM at ∼100× higher latency. In contrast, S_{diff} sidesteps the regression problem entirely: the coarse signal suffices because Phase 3 amplifies even an approximate difficulty ranking into system-level QA-Goodput gains. Moreover, S_{diff} serves as a proxy for the decode cost $\hat{D}_i(q_i)$ in Eq. 4: queries requiring more bridging evidence tend to produce longer reasoning chains [9], so the same topological signal that guides budget allocation also approximates GPU occupancy duration. The scoring is *zero-cost and training-free*: all three features are byproducts of Phase 1 retrieval (< 2 ms CPU overhead), and w is calibrated by grid search on held-out data without task-specific training.

4.4.3 Dynamic Budget Allocation via Adaptive K. Given $S_{\text{diff}}(q)$, the controller must translate difficulty into a concrete budget K_q. The design must satisfy two requirements: (R1) maximize answer quality when capacity is abundant, and (R2) gracefully degrade under congestion without erasing difficulty-based differentiation.

Quality-first regime. When the system is uncongested (queue utilization $u(t) \le \theta_{\text{bp}}$), the budget is driven purely by difficulty:

$$K_q = K_{\min} + \lfloor (K_{\max} - K_{\min}) \cdot S_{\text{diff}}(q) \rceil. \tag{6}$$

We denote this quality-first budget as $K_{\text{diff}}(q)$; it serves as the candidate pool size for context assembly (Section 4.4.4). In Example 5, q' receives $K_{q'} \approx K_{\max}$ (full retrieval depth, retaining the inventory-lock evidence) while q'' receives $K_{q''} \approx K_{\min}$ (the single order-record chunk suffices), saving prefill tokens that would otherwise be wasted. The linear mapping is deliberate: under Zipf-distributed workloads where popular (easy) queries dominate, linearity produces a smooth, unimodal K distribution that avoids the bimodal pathology of nonlinear mappings [19].

Congestion regime and proportional degradation. We formalize the congestion signal introduced in the cost model (Eq. 4). The instantaneous queue utilization $u(t) \in [0, 1]$ is the ratio of pending prefill workload to the scheduling window capacity:

$$u(t) = \frac{\sum_{i \in \mathcal{Q}(t)} p_i}{\delta}, \tag{7}$$

where $\mathcal{A}(t) \in \mathcal{Q}$ is the set of requests awaiting or undergoing prefill at time t, $p_i = L_i(K_i)/\mu$ is the prefill time defined in Eq. 4, and δ is the scheduling window length. When Phase 3 signals congestion ($u(t) > \theta_{\text{bp}}$), the controller must reduce the aggregate prefill load while preserving difficulty-based differentiation. We introduce a *load-dependent effective ceiling* $K_{\text{eff}}(t)$: a global variable, recomputed once per scheduling window from the serving engine's queue utilization $u(t)$, that caps the maximum budget any single query may receive. $K_{\text{eff}}(t)$ must satisfy three boundary conditions: **(B1)** $K_{\text{eff}} = K_{\max}$ when the system is uncongested ($u(t) \le \theta_{\text{bp}}$); **(B2)** $K_{\text{eff}} \to K_{\min}$ as $u(t) \to 1$ (full saturation); **(B3)** K_{eff} is monotonically decreasing in $u(t)$, so that heavier load always reduces the budget ceiling.

Connection to Online Knapsack. Choosing $K_{\text{eff}}(t)$ under stochastic arrivals is an instance of the *online knapsack problem* [53]: each arriving query q is an item with weight K_q (prefill tokens consumed) and value $Q(K_q, q)$ (answer quality contributed); the GPU's prefill capacity per scheduling window is the knapsack capacity. Queries arrive one at a time, and the controller must irrevocably decide each query's budget before observing future arrivals. The design of $K_{\text{eff}}(t)$ corresponds to the *reservation function* (admission threshold) in this framework: as utilization $u(t)$ grows, the system raises its admission bar by lowering the maximum per-query budget. The question is: what *shape* should this threshold take?

Theorem 3: *[Optimal Reservation for Online Budget Selection] Consider a stream of items arriving online, each with value-to-weight ratio $v \in [L, U]$, and a knapsack of capacity C.* **(a)** *No deterministic online algorithm achieves a competitive ratio better than* $\ln(U/L) + 1$ [53]. **(b)** *The exponential threshold $\Psi(z) = L \cdot (U/L)^z$, which admits an item iff $v \ge \Psi(z)$ at utilization $z \in [0, 1]$, achieves this bound and is therefore optimal* [8]. **(c)** *A linear threshold achieves only $O(\sqrt{U/L})$, which is strictly worse when $U/L > e$* [44]. □

Proof sketch. Part (a): an adversary reveals high-value items late, forcing any fixed threshold to either over-commit early or reject late [53]. Part (b): the exponential equalizes marginal competitive loss across all utilization levels [8]. Full proof in Appendix A.5. □

Instantiation. In DynaGraph's setting, U/L maps to $K_{\max}/K_{\min}$ (the ratio of maximum to minimum per-query budget). Applying Theorem 3(b), we instantiate $K_{\text{eff}}(t)$ as:

$$K_{\text{eff}}(t) = \begin{cases} K_{\max} & \text{if } u(t) \le \theta_{\text{bp}}, \\ K_{\min} + (K_{\max} - K_{\min}) \cdot \exp\left(-\alpha_{\text{bp}} \cdot \dfrac{u(t) - \theta_{\text{bp}}}{1 - \theta_{\text{bp}}}\right) & \text{otherwise,} \end{cases} \tag{8}$$

with $\alpha_{\text{bp}} = \ln(K_{\max}/K_{\min})$. This satisfies all three boundary conditions by construction: (B1) the piecewise guard; (B2) α_{bp} is chosen so that $K_{\text{eff}} \to K_{\min}$ as $u(t) \to 1$; (B3) the exponential is monotonically decreasing. The shift by θ_{bp} defines a "free zone" where the

system operates without capacity pressure, a standard extension in online algorithms with initial slack [7]. Each arriving query then receives a per-query budget proportional to its difficulty within the current ceiling:

$$K_q = K_{\min} + \lfloor (K_{\text{eff}}(t) - K_{\min}) \cdot S_{\text{diff}}(q) \rceil. \tag{9}$$

$K_{\text{eff}}(t)$ governs the *system-level* ceiling, while $S_{\text{diff}}(q)$ governs the *per-query* allocation within $[K_{\min}, K_{\text{eff}}(t)]$. This *proportional degradation* compresses the budget range under load rather than collapsing all queries to a single value, ensuring that q' still receives more context than q'' even during congestion (Example 5). When $u(t) \le \theta_{\text{bp}}$, Eq. (9) reduces to the quality-first mode (Eq. (6)), so Eq. (9) serves as the unified control law for both regimes.

The interplay of arrival rate λ and $K_{\text{eff}}(t)$ yields three operating regimes: (i) *quality-first* ($u(t) < \theta_{\text{bp}}$, backpressure inactive); (ii) *transition* ($u(t) \approx \theta_{\text{bp}}$, mode switching); (iii) *goodput-first* ($u(t) \gg \theta_{\text{bp}}$, exponential decay dominates). A QPS sweep experiment validates all three regimes (Section 5.7); Figure 9 provides a per-request runtime trace showing how $K_{\text{eff}}(t)$ tracks queue utilization while preserving difficulty-based differentiation.

Remark. Adaptive K and priority scheduling are complementary, not substitutable. Phase 2 alone reduces token cost but cannot reorder requests: under FCFS, a burst of hard queries still causes head-of-line blocking (Obs. 2). Phase 3 alone reorders requests but cannot reduce their cost: with $K = K_{\max}$ for every query, the scheduler has no cost variance to exploit (Obs. 3). Their combination creates cost heterogeneity ($K_{\text{easy}} \ll K_{\text{hard}}$) that the scheduler interleaves, while congestion triggers backpressure via Eq. (8)—a feedback loop whose super-additive effect is quantified in Section 5.

4.4.4 Context Assembly. Given K_q from Eq. (9), the assembler selects K_q evidence units from the candidate corpus C_q and constructs the final prompt.

Multi-Source Candidate Scoring. Phase 1 produces three candidate pools with different provenance: embedding-retrieved chunks (C_q^{chunk}), their parent documents (C_q^{doc}), and graph-traversal passages (C_q^{graph}). After deduplication, every candidate $d \in C_q$ receives a unified relevance score:

$$s(d, q) = s_{\text{rerank}}(d, q) + \delta \cdot \mathbb{I}\big(d \in C_q^{\text{graph}}\big), \tag{10}$$

where s_{rerank} is the cross-encoder score and δ is a graph-presence bonus. This merges heterogeneous candidates into a single ranked list $\mathcal{D}_q$.

Subset Selection. When uncongested ($K_q = K_{\text{diff}}(q)$), the assembler truncates $\mathcal{D}_q$ at position K_q. Under backpressure ($K_q < K_{\text{diff}}(q)$), naïve truncation may discard complementary evidence scattered across entity clusters—precisely the setting where bridge queries suffer most. We instead select K_q representatives from the quality-first pool $\mathcal{D}_q[1{:}K_{\text{diff}}]$ by solving a *relevance-weighted k-center* problem in the embedding space already computed by Phase 1:

$$\mathcal{S}_q^* = \arg \min_{|\mathcal{S}|=K_q} \max_{d \in \mathcal{D}_q[1:K_{\text{diff}}]} s(d, q) \cdot \big(1 - \max_{d' \in \mathcal{S}} \cos(\mathrm{e}_d, \mathrm{e}_{d'})\big), \tag{11}$$

where e_d is the document embedding from Phase 1 and $\cos(\cdot, \cdot)$ denotes cosine similarity. The objective encourages *diversity*: the cosine-distance term penalizes selecting documents that are redundant in embedding space, while the relevance weight $s(d, q)$ ensures that high-importance candidates are prioritized for coverage.

This is a weighted k-center problem, solved greedily by farthest-first traversal [16] with a 2-approximation guarantee. Since the candidate pool contains at most $|C_q|$ documents (typically $\le$50), the greedy solver requires $O(|C_q| \cdot K_q)$ cosine evaluations—executed entirely on CPU in microseconds without blocking the GPU inference pipeline. The selected documents are returned in their original $s(d, q)$ order. The final prompt is $P_q = [\, I_{\text{sys}},\ \mathcal{S}_q,\ q \,]$, and the resulting length $L_q = |P_q|$ is passed to Phase 3 as the estimated prefill cost.

Remark. The budget decision and the document selection rely on different signal types by design. Graph-topological features (*e.g.,* bridge ratio, overlap density) capture *global* structural complexity—they answer "how hard is this query?" but are agnostic to which individual documents carry answer-relevant content. Document embeddings, by contrast, capture *local* semantic position—they answer "how similar or complementary are these two passages?" but say nothing about overall query difficulty. DynaGraph exploits this complementarity: topological signals govern the budget ($S_{\text{diff}} \rightarrow K_q$), while embedding-space coverage governs document selection ($\mathcal{D}_q \rightarrow \mathcal{S}_q$). This separation of concerns avoids conflating two orthogonal objectives and allows each signal to operate in the regime where it is most informative.

4.5 Phase 3: SLO-Aware Scheduling (The Actuator)

Phase 2 delivers a stream of optimized prompts $\{P_q\}$, each with prefill time $p_q = L_q/\mu_{\text{p}}$ (Eq. 4) and difficulty score $S_{\text{diff}}(q)$. Under light load, the order in which prompts reach the LLM engine matters little. However, under bursty traffic, head-of-line (HOL) blocking becomes the dominant bottleneck (Obs. 2). A few supernode-dominated queries with $L_q > 10{,}000$ tokens stall all subsequent requests, cascading SLO misses even to lightweight queries.

Phase 3 replaces the default FCFS policy with a **multi-signal priority scheduler** that jointly considers SLO urgency, cost efficiency, and graph-derived structural priors. We implement the scheduler on top of vLLM's [30] priority API; the design is engine-agnostic and applicable to other frameworks such as SGLang [51].

Example 6:[SLO-Aware Scheduling] Under a common SLO deadline τ=500 ms:

- **Urgent query:** The customer query q' ("payment succeeded but order failed") requires K=15 chunks ($L_q \approx$ 8,000 tokens), consuming $p_q/\tau \approx 0.9$ of its SLO budget. The scheduler assigns high urgency.
- **Relaxed query:** A simple entity lookup needs only K=5 chunks ($L_q \approx$ 1,500 tokens, $p_q/\tau \approx 0.17$). It can wait without SLO risk.
- **Tie-breaking:** Among equally urgent requests, the scheduler favors lower L_q, freeing GPU resources sooner.
- **Fairness:** The backpressure mechanism provides a natural safeguard (Section 4.5.3).

□

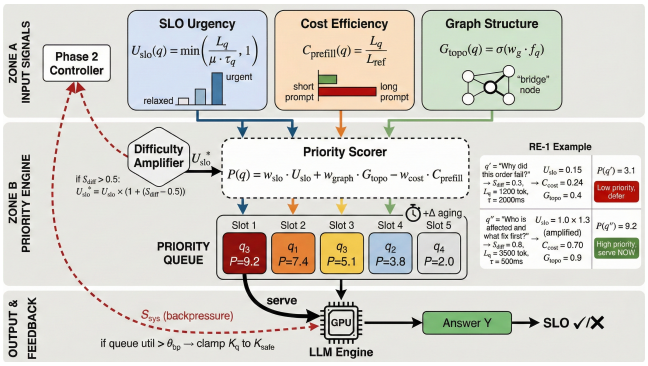


Figure 7: SLO-Aware Scheduling Logic. The scheduler aggregates three signals derived from the cost model (Eq. 4): SLO urgency U_{slo}, cost efficiency C_{prefill}, and structural prior G_{topo}. The resulting priority $\mathcal{P}(q)$ orders the execution queue, while a feedback monitor emits S_{sys} when utilization spikes.

4.5.1 Priority Function Design. Phase 3 solves the scheduling sub-problem of ($\mathcal{P}$) (Eq. 3): given the budgets K from Phase 2, choose the permutation π that maximizes $\mathcal{J}_\lambda$. Since this sub-problem is NP-hard (Theorem 2), we adopt a greedy strategy. The cost model (Eq. 4) suggests three scheduling signals, one from each dimension: *Dimension 1* (prefill cost, first term of Eq. 4) yields two complementary views of each query's resource footprint: (i) *SLO urgency* p_q/τ_q, measuring how much of the deadline the prefill alone consumes, and (ii) *cost efficiency* L_q/L_{ref}, penalizing expensive requests that cause HOL blocking. *Dimension 2* (reasoning complexity S_{diff}, which also approximates the decode cost $\hat{D}_i(q_i)$ in the second term of Eq. 4; see Remark in Section 4.4.2) yields a *topological prior* that distinguishes structurally complex queries before runtime telemetry is available. This three-signal design parallels the classical tension between SJF, EDF, and priority scheduling in operating systems [39].

SLO Urgency Signal (U_{slo}). Using the prefill time $p_q = L_q/\mu_p$ from the cost model (Eq. 4):

$$U_{\text{slo}}(q) = \min\left(\frac{p_q}{\tau_q},\ 1\right), \tag{12}$$

where τ_q is the TTFT deadline. A value near 1 signals that the prefill alone nearly exhausts the SLO budget.

Cost Efficiency Signal (C_{prefill}). To mitigate HOL blocking, we penalize prompts with high token counts, normalized by a reference ceiling L_{ref}:

$$C_{\text{prefill}}(q) = \frac{L_q}{L_{\text{ref}}}. \tag{13}$$

Deprioritizing expensive requests during congestion implements SJF scheduling [27], which minimizes average waiting time.

Graph Structure Signal (G_{topo}). Phase 1's topological features provide an initial estimate of each query's structural importance:

$$G_{\text{topo}}(q) = \sigma\left(\mathrm{w}_g^\top \cdot \tilde{\mathrm{f}}_q\right), \tag{14}$$

where $\tilde{\mathrm{f}}_q$ is the normalized topological feature vector from Phase 1 and w_g is a learned weight vector. G_{topo} and S_{diff} (Eq. 5) both derive from Phase 1's topological features $\tilde{\mathrm{f}}_q$ ($r_{\text{bridge}}, \rho_{\text{overlap}}, n_{\text{entity}}$), but with different learned weights reflecting their distinct roles: S_{diff} emphasizes ρ_{overlap} to estimate how much context a query needs, guiding Phase 2's budget allocation (K_q, Eq. 9), while G_{topo} emphasizes r_{bridge} to estimate how urgently the query should be served, guiding Phase 3's priority ($\mathcal{P}(q)$, Eq. 15), since queries spanning multiple bridging entities carry higher cascading-miss risk.

Composite Priority. The final priority combines the three signals:

$$\mathcal{P}(q) = w_{\text{slo}} \cdot U^*_{\text{slo}}(q) + w_{\text{graph}} \cdot G_{\text{topo}}(q) - w_{\text{cost}} \cdot C_{\text{prefill}}(q), \tag{15}$$

where $\{w_{\text{slo}}, w_{\text{graph}}, w_{\text{cost}}\}$ are non-negative mixing weights. Requests with higher $\mathcal{P}(q)$ are served first. The positive U_{slo} term promotes urgent requests (EDF-like); the negative C_{prefill} term suppresses expensive ones (SJF-like); the G_{topo} term breaks ties in favor of structurally important queries. The weights are calibrated via grid search on held-out data, consistent with Phase 2's weight calibration (Section 4.4.3).

Remark (Connection to OS Scheduling). DynaGraph's composite priority unifies three classical OS scheduling principles [39]: C_{prefill} implements SJF [27], U_{slo} implements EDF [17], and Phase 2's adaptive K dynamically modulates job "size" while Phase 3's backpressure closes the feedback loop, analogous to MLFQ [2]. Unlike generic OS schedulers, however, DynaGraph incorporates graph-topology signals and cross-phase feedback; the resulting improvements over pure SJF/EDF baselines are quantified in Section 5.4.

Remark (Graph-Informed Scheduling). Existing LLM schedulers [30, 51] and RAG systems [24] treat all requests as opaque token sequences with homogeneous cost. By incorporating G_{topo}, DynaGraph's scheduler becomes topology-aware: it can distinguish a short prompt covering a single-hop lookup from one bridging multiple evidence clusters. The latter warrants earlier service even at moderate L_q, because its downstream impact on QA quality is disproportionately high.

Approximation Guarantee. **Assumption 4:***[Independent Evidence]*

Serving query q_i within its deadline contributes $v_i = Q(K_i, q_i)$ to QA-Goodput independently of whether other queries meet their deadlines. This follows from the additive structure of $\mathcal{J}_\lambda$ (Eq. 2): total goodput is the sum of per-query contributions over the on-time set S, i.e., $\mathcal{J}_\lambda = \frac{1}{n}\sum_{i\in S} v_i$. No inter-query quality coupling is assumed. □

Theorem 5:*[Scheduling Approximation] Under Assumption 4 and common deadline τ:*

(*i*) *The greedy algorithm that processes queries in decreasing value-density v_i/p_i and takes the better of the greedy packing and the best single item achieves $\mathcal{J}_{\text{greedy}} \geq \frac{1}{2} \cdot \text{OPT}$. No assumption on the processing-time distribution is required.*

(*ii*) *Let $r(t) = K_{\text{eff}}(t)/K_{\min}$ be the processing-time ratio induced by Phase 2's budget allocation (Theorem 3). The greedy algorithm additionally satisfies $\mathcal{J}_{\text{greedy}} \geq \frac{1}{1+r(t)} \cdot \text{OPT}$. Since $K_{\text{eff}}(t)$ decreases under load (Eq. 8), $r(t)$ shrinks under congestion, strengthening the bound: from $\geq\frac{1}{3}$ (quality-first, r=2) toward optimality as $r(t) \to 1$ (goodput-first).*

□

Proof Sketch. (i) With common deadline τ, feasible scheduling reduces to 0-1 knapsack (capacity τ, item weights p_i, values v_i). The 1/2-bound follows from Dantzig's fractional relaxation [12]. (ii) Phase 2 bounds $K_i \in [K_{\min}, K_{\text{eff}}(t)]$, so the processing-time ratio is at most $r(t)$. The feasible subsets form an $r(t)$-system [29], for which greedy achieves $1/(1+r(t))$. Full proof in Appendix A.2. □

Remark (Phase 2–3 Synergy). Theorem 5(ii) reveals a theoretical complementarity: Phase 2's backpressure reduces $r(t)$ under congestion, tightening Phase 3's approximation guarantee precisely when scheduling quality matters most.

4.5.2 Difficulty-Aware Compensation. The greedy scheduler of Section 4.5.1 addresses HOL blocking (Obs. 2), but has two residual limitations. First, it is *difficulty-blind*: two queries with identical L_q receive the same priority, yet a hard multi-hop query whose late answer is operationally useless [9] deserves higher urgency than a simple lookup that can be retried. Second, it is *load-blind*: priority reordering can only redistribute latency, not reduce total load; when the arrival rate exceeds capacity, the queue grows unboundedly regardless of scheduling quality (Obs. 2). DynaGraph addresses both limitations through two complementary mechanisms that leverage the decode-cost dimension of the cost model (second term of Eq. 4).

Forward Compensation: Urgency Amplification. Phase 2's difficulty score $S_{\text{diff}}(q)$ is forwarded into the priority function to modulate SLO urgency. We define the amplified urgency as $U^*_{\text{slo}}(q) = U_{\text{slo}}(q) \cdot \varphi(S_{\text{diff}}(q))$, where $\varphi : [0,1] \to [\varphi_{\min}, \varphi_{\max}]$ is a monotonically increasing modulation function. We instantiate φ as a piecewise-linear function: $\varphi(s) = 1$ for $s \le 0.5$ and $\varphi(s) = 1 + (s - 0.5)$ for $s > 0.5$, granting up to 1.5× urgency amplification for the hardest queries. This completes the end-to-end information flow: Phase 1 extracts topological features $\mathcal{F}_q$, Phase 2 computes S_{diff} and uses it to set K_q, and Phase 3 reuses S_{diff} to modulate scheduling urgency.

Backward Compensation: Backpressure Signal. When scheduling alone cannot prevent queue buildup, Phase 3 sends a congestion signal back to Phase 2. The scheduler monitors queue utilization $u(t) \in [0,1]$ and emits the backpressure signal

$$S_{\text{sys}}(t) = \mathbb{I}\big(u(t) > \theta_{\text{bp}}\big), \tag{16}$$

where θ_{bp} is the backpressure threshold defined in Section 4.4.3. Upon receiving $S_{\text{sys}} = 1$, Phase 2's controller activates proportional degradation (Eqs. 8–9): $K_{\text{eff}}(t)$ decays exponentially with utilization, compressing budgets into $[K_{\min}, K_{\text{eff}}(t)]$. The reduced L_q in turn lowers U_{slo} and C_{prefill} for new arrivals, naturally alleviating queue pressure. This forms a negative feedback loop [19]: utilization rises → budgets shrink → prefill shortens → queue drains → utilization falls. Unlike reactive approaches such as DepCache [47] that prune attention at inference time, backpressure acts *proactively*, reducing context volume before the request enters the engine.

Queue Stability. We now establish that the backpressure mechanism guarantees bounded queue length under bursty arrivals. In the stability analysis, the design variables map to theorem variables as follows: $\lambda_{\max}$ is the MMPP peak arrival rate, $K_{\min} = \lim_{u\to 1} K_{\text{eff}}(t)$ is the worst-case budget under full congestion, and μ_{p} is the prefill throughput from the cost model (Eq. 4).

Theorem 6: *[Queue Stability under MMPP Arrivals] Discretize time into slots of duration δ. Let arrivals follow an MMPP with peak rate $\lambda_{\max}$ and per-query service rate μ_p. Under Phase 2's backpressure controller, which enforces $K_q \le K_{eff}(t)$ with $K_{eff}(t) \to K_{\min}$ as $u(t) \to 1$ (Eq. 8), the queue is positive recurrent whenever:*

$$\lambda_{\max} \cdot \frac{K_{\min}}{\mu_p} < 1. \tag{17}$$

□

Proof Sketch. Define $V(t) = \frac{1}{2}Q(t)^2$. Under sustained congestion, Phase 2 drives $K_{\text{eff}}(t) \to K_{\min}$, bounding every query's service time by $K_{\min}/\mu_{\text{p}}$. The one-slot drift satisfies $\mathbb{E}[\Delta V \mid Q(t)] \le Q(t)(\lambda_{\max} K_{\min}/\mu_{\text{p}} - 1) + B$, which is strictly negative for large $Q(t)$ when Eq. 17 holds. Positive recurrence follows by Foster–Lyapunov [33]. See Appendix A.3. □

Theorem 6 yields a **closed-form design rule**: the system is stable whenever $K_{\min} < \mu_{\text{p}}/\lambda_{\max}$, a condition verified once at deployment time.

4.5.3 Fairness. Priority-based scheduling may in principle starve low-priority requests. In DynaGraph, backpressure provides a natural safeguard: under sustained congestion, Phase 2 compresses all budgets toward $K_{\min}$ (Eq. 8), equalizing processing times and preventing any single query class from monopolizing GPU resources. Experimentally, P99 TTFT remains bounded across all configurations (Table 1). If additional guarantees are needed, a standard aging mechanism [39] can be layered on: $\mathcal{P}_{\text{aged}}(q) = \mathcal{P}(q) + \Delta_{\text{age}} \cdot \max(0,\ (t_{\text{wait}}(q) - T_{\text{age}})/\tau_q)$, guaranteeing bounded expected wait; the formal bound is derived in Appendix A.4.

Output Summary. Phase 3 produces:

- **Execution Order:** An optimized serving sequence that maximizes $\mathcal{J}_\lambda$ (Section 2.2).
- **Inference Results** Y**:** The autoregressively generated answers.
- **Backpressure Signal** S_{sys}**:** Continuous congestion feedback to Phase 2.

4.6 From Phases to a Closed-Loop System

Algorithm 1 formalizes the complete data flow. The three phases form a **closed-loop control system**, not a linear pipeline.

The red entries highlight the feedback channel: S_{sys} flows backward from Phase 3 to Phase 2, closing the loop. Forward signals ($\mathcal{F}_q \to K_q \to \mathcal{P}(q)$) resolve retrieval blindness; the backward signal ($S_{\text{sys}} \to K_q$) resolves serving agnosticism.

Remark (Complementarity). A central finding of our motivational study (Table 1) is that the phases are complementary: adaptive K alone achieves 45.9% SLO@200ms; scheduling alone only 21.6%; the joint system reaches 74.4%. The forward path (P1→P2) primarily optimizes Quality vs. Cost; the backward path (P3→P2) primarily optimizes Latency vs. Throughput. Neither alone is sufficient; their interaction produces a super-additive improvement.

Algorithm 1: DynaGraph Closed-Loop Pipeline

Input : Query stream $\{q\}$, knowledge graph $\mathcal{G}$, SLO deadlines $\{\tau_q\}$
Output: Answer stream $\{Y_q\}$

$S_{\text{sys}} \leftarrow 0$ // init: no congestion
foreach *incoming query* q **do**
  /* Phase 1: Retrieval (Sensor) */
  $\mathcal{G}_q, \mathcal{F}_q \leftarrow \text{Retrieve}(q, \mathcal{G})$
  /* Phase 2: Adaptive Aggregation (Controller) */
  $S_{\text{diff}} \leftarrow \sigma(\mathrm{w}^\top \tilde{\mathrm{f}}_q + b)$ // Eq. 5
  $K_{\text{eff}} \leftarrow$ Eq. 8 using $u(t)$
  $K_q \leftarrow K_{\min} + \lfloor (K_{\text{eff}} - K_{\min}) \cdot S_{\text{diff}} \rceil$ // Eq. 9
  $P_q \leftarrow \text{Assemble}(\mathcal{G}_q, K_q)$
  /* Phase 3: SLO-Aware Scheduling (Actuator) */
  $U^*_{\text{slo}} \leftarrow U_{\text{slo}} \cdot \varphi(S_{\text{diff}})$ // urgency amplification
  $\mathcal{P}(q) \leftarrow w_{\text{slo}} U^*_{\text{slo}} + w_{\text{graph}} G_{\text{topo}} - w_{\text{cost}} C_{\text{prefill}}$ // Eq. 15
  Insert q with key $\mathcal{P}(q)$
  $Y_q \leftarrow \text{ServeHighestPriority}()$
  /* Feedback Channel */
  $u(t) \leftarrow \text{QueueUtilization}()$
  $S_{\text{sys}} \leftarrow \mathbb{I}(u(t) > \theta_{\text{bp}})$ // Eq. 16
end

Table 4: Phase Summary. Input–output relationships and system-level roles. Red entries highlight the feedback channel.

Phase	Role	Input	Output
P1: Retrieval	Sensor	$q, \mathcal{G}$	$\mathcal{G}_q, \mathcal{F}_q$
P2: Aggregation	Controller	$\mathcal{G}_q, \mathcal{F}_q, S_{\text{sys}}$	$P_q, L_q, S_{\text{diff}}$
P3: Scheduling	Actuator	$P_q, L_q, S_{\text{diff}}, \tau_q$	Y, S_{sys}

5 EXPERIMENTAL EVALUATION

We design experiments to answer the following research questions:

- **RQ1**: Does DynaGraph outperform existing GraphRAG baselines in both answer quality and serving efficiency?
- **RQ2**: What is the contribution of each phase (P2, P3) and their cross-layer feedback?
- **RQ3**: How do different scheduling strategies affect serving performance?
- **RQ4**: How does DynaGraph's difficulty-aware adaptive-K compare against alternative context selection methods?
- **RQ5**: How do Phase 2's internal design choices (decay shape, feature selection, $k_{\min}$) affect the quality–latency trade-off?
- **RQ6**: How robust is DynaGraph under varying load?
- **RQ7**: Does DynaGraph remain effective under end-to-end online deployment with live retrieval?

5.1 Setup

Datasets. We evaluate on three established multi-hop QA benchmarks: **2WikiMultihopQA** [20] (2Wiki, 12,576 validation queries), **HotpotQA** [45] (7,405 validation queries), and **MuSiQue** [41] (2,417 validation queries). Following the evaluation protocol established by HippoRAG [18] and adopted by subsequent works [25, 35, 49], we sample 1,000 queries from each validation set with their associated supporting and distractor passages as the retrieval corpus (~11K passages per dataset). For scalability analysis (RQ7), we additionally evaluate on the full 2Wiki validation set (12,576 queries, ~57K passages).

Models. We use two LLMs: **Qwen3-4B** (pure-attention, single RTX 4090) and **Qwen3.5-4B** (hybrid Mamba, 2×RTX 4090 TP=2). Embedding: Qwen3-Embedding-0.6B; reranking: bge-reranker-v2-m3.

Infrastructure. 2×NVIDIA RTX 4090 (24 GB each), 128 GB RAM. LLM serving: **vLLM 0.17/0.18** with prefix caching, chunked prefill, and fp8 KV cache. Online workloads: Zipf(α=1.0) at 30 QPS (3,000 total requests).

Metrics. **Answer quality**: token-level F1 and substring-match Accuracy (Acc), following standard multi-hop QA evaluation [41, 45]. **Latency**: Time-to-First-Token (TTFT) at p50/p95/p99, the standard responsiveness metric in LLM serving [30, 52]. **SLO attainment**: fraction of requests with TTFT $\leq \tau$, widely used to quantify service reliability [1, 52]. **QA-Goodput**: $\mathcal{J}_\lambda = \frac{1}{N} \sum_i F_1^{(i)} \cdot \mathbb{I}(\text{TTFT}_i \leq \tau)$, extending the throughput-oriented Goodput [52] with answer-quality weighting to jointly capture accuracy and timeliness. We report $\mathcal{J}$ at $\tau \in \{200, 500, 2000\}$ ms to cover strict to relaxed SLO regimes. RQ1–RQ6 report LLM serving TTFT only (all methods share identical pre-computed retrieval); RQ7 reports end-to-end TTFT with live retrieval.

5.2 RQ1: End-to-End System Comparison

Table 5 compares DynaGraph against four published GraphRAG methods under online serving on all three datasets, plus VanillaGRAG †—a no-optimization variant using the same Phase 1 retrieval but always concatenating the union of all hop-2 graph documents and top corpus candidates (up to K=50) without adaptive-K or priority scheduling. All methods use the same LLM (Qwen3.5-4B) and serving engine (vLLM, FCFS). Each baseline uses its own retrieval pipeline and prompt template.

Key findings. (1) DynaGraph achieves the highest F1 among all external baselines on all three datasets (0.787 / 0.769 / 0.471), outperforming the best prior work by 9–27% relative F1. (2) Long-context methods (SiReRAG 5–7k tokens, KET – RAG 12k) achieve 0% SLO@500ms on all datasets—prompts too long for online serving at 30 QPS. (3) HippoRAG meets SLO (70–94%) with ~600 tokens, but sacrifices multi-hop quality (F1=0.178 on MuSiQue vs our 0.471). (4) DynaGraph uniquely achieves both the highest SLO (≥94.9%) and highest $\mathcal{J}$@500 across all datasets, nearly doubling HippoRAG's GP (72.8% vs 38.8% on 2Wiki, 47.6% vs 16.1% on MuSiQue). (5) VanillaGRAG ($K \leq 50$) isolates the Phase 2/3 contribution: using DynaGraph's Phase 1 graph retrieval but always serving 50 documents achieves F1 = .768 / .758 / .501 on 2Wiki / HotpotQA / MuSiQue, yet SLO@500ms of only 37% / 36% / 49%—a 2–4× serving gap vs DynaGraph (94.9% / 95.5% / 97.7%). On 2Wiki and HotpotQA, DynaGraph *improves* F1 by 1–2pp while closing the SLO gap, because difficulty-aware K selection removes document noise for easy queries without truncating evidence for hard ones. On MuSiQue,

Table 5: End-to-end system comparison under online serving (Qwen3.5-4B, 1000 unique queries, Zipf@30qps, 3000 total requests). † = our reproduction. $\mathcal{J}=\frac{1}{N}\sum_i F_1^{(i)}\cdot\mathbb{I}[\mathbf{TTFT}_i \le \tau]$.

Method	F1↑	Acc↑	Toks	p50↓	SLO@500↑	SLO@200↑	$\mathcal{J}$@500↑	$\mathcal{J}$@200↑
2WikiMultihopQA								
SiReRAG † [49]	.721	.728	6760	8629	0.0%	0.0%	0.0%	0.0%
KET – RAG † [35]	.470	.438	11833	19255	0.0%	0.0%	0.0%	0.0%
ClueRAG † [25]	.637	.644	1165	788	6.8%	0.1%	4.8%	0.1%
HippoRAG † [18]	.410	.489	554	355	93.9%	2.9%	38.8%	1.0%
VanillaGRAG † ($K\le50$)	.768	.840	8339	582	37.1%	28.8%	32.9%	25.7%
DynaGraph	**.787**	**.841**	**2391**	**220**	**94.9%**	**45.5%**	**72.8%**	**37.5%**
HotpotQA								
SiReRAG †	.570	.483	5482	6346	0.0%	0.0%	0.0%	0.0%
KET – RAG †	.603	.524	11613	19144	0.0%	0.0%	0.0%	0.0%
ClueRAG †	.512	.455	709	725	5.8%	0.1%	3.3%	0.0%
HippoRAG †	.507	.522	620	426	70.4%	1.5%	35.6%	0.6%
VanillaGRAG † ($K\le50$)	.758	.718	8302	570	36.4%	28.6%	20.7%	15.3%
DynaGraph	**.769**	**.716**	**2244**	**224**	**95.5%**	**40.3%**	**53.3%**	**21.5%**
MuSiQue								
SiReRAG †	.427	.266	6712	7751	0.0%	0.0%	0.0%	0.0%
KET – RAG †	.301	.152	11909	19060	0.0%	0.0%	0.0%	0.0%
ClueRAG †	.409	.250	904	715	6.8%	0.2%	3.1%	0.0%
HippoRAG †	.178	.169	667	345	89.5%	5.1%	16.1%	1.1%
VanillaGRAG † ($K\le50$)	.501	.549	7876	510	48.5%	37.6%	29.8%	22.3%
DynaGraph	**.471**	**.507**	**2088**	**173**	**97.7%**	**63.6%**	**47.6%**	**32.8%**

Table 6: Serving baseline comparison with fixed retriever (DynaGraph P1, 2Wiki, 1000q×3000, Zipf@30qps). ‡ = requires non-standard attention layout.

Method	K	Sched	F1↑	Acc↑	p50↓	SLO@500↑	$\mathcal{J}$@500↑
Qwen3-4B (lightly loaded)							
C1 (FCFS K=20)	Fixed 20	FCFS	.762	.793	288	85.1%	69.8%
CacheBlend ‡ [46]	Fixed 20	FCFS	.667	.725	18295	0.0%	0.0%
DynaGraph (C5)	Joint	+BP	**.753**	**.797**	**143**	**97.9%**	**78.6%**
Qwen2.5-7B (medium loaded)							
C1 (FCFS K=20)	Fixed 20	FCFS	.546	.579	301	75.9%	52.4%
CacheBlend ‡ [46]	Fixed 20	FCFS	.564	.595	11034	0.0%	0.0%
DepAttn ‡ [47]	Fixed 20	FCFS	.058	—	213	96.3%	5.6%
DynaGraph (C5)	Joint	+BP	**.602**	**.621**	**144**	96.2%	**64.8%**

Table 7: Phase ablation on Qwen3.5-4B (2Wiki, Zipf@30qps, heavily loaded—FCFS collapses). KV-cache baselines cannot run on this hybrid Mamba architecture, highlighting DynaGraph's model-agnostic design.

Config	K	Sched	F1↑	Acc↑	p50↓	SLO@500↑	$\mathcal{J}$@500↑
C1 (Static)	Fixed 20	FCFS	.800	.857	5892	0.0%	0.0%
C3 (P2 only)	Diff	FCFS	.787	.844	4076	0.0%	0.0%
B4 (P3 only)	Fixed 20	Comp.	.793	.859	407	57.8%	41.6%
C5 (Full)	Joint	+BP	.787	.841	**220**	**94.9%**	**72.8%**

where deep four-hop chains require the full 50-document budget, DynaGraph incurs a modest −3pp F1 trade-off (0.471 vs 0.501) to achieve +49pp SLO (97.7% vs 48.5%)—the exact quality-cost frontier identified in Section 3.

5.3 RQ2: Serving Strategy Comparison

To isolate the contribution of serving-level optimizations, we fix the retriever to DynaGraph's P1 module and compare against KV-cache optimization methods on 2WikiMultihopQA at Zipf@30qps. Table 6 compares against external baselines; Table 7 ablates DynaGraph's internal phases.

External baselines (Table 6). (1) CacheBlend [46] fuses precomputed per-chunk KV caches with selective recomputation. Under 30 QPS concurrent load, the fusion overhead causes TTFT to balloon to 18 s on Qwen3-4B and 11 s on Qwen2.5-7B (0% SLO@500ms in both cases)—the method is designed for single-request settings and cannot sustain high-concurrency online serving. (2) DepAttn [47] applies block-diagonal attention to enable per-chunk KV reuse. It achieves low TTFT (213 ms) from prefix sharing, but F1 collapses from 0.546 to 0.058 (−49pp) because the block-diagonal mask prevents cross-document reasoning essential for multi-hop QA. The pattern holds across all datasets (HotpotQA: 0.55→0.07; MuSiQue: 0.34→0.03). (3) DynaGraph achieves both the lowest TTFT (143/144 ms) and highest $\mathcal{J}$@500 (78.6%/64.8%) across both models, confirming that adaptive context selection (Phase 2) and priority scheduling (Phase 3) address a fundamentally different axis from KV-cache reuse—and are more effective for GraphRAG workloads.

Availability of other serving baselines. RAGCache [26], FusionRAG [43], and METIS [38] are not open-sourced at submission time. RAGCache and FusionRAG optimize KV-cache reuse (the same axis as CacheBlend); METIS performs per-query RAG configuration adaptation, which is complementary to DynaGraph's topological scoring. All KV-cache methods require standard self-attention layouts and cannot be applied to hybrid architectures (e.g., Qwen3.5-4B's Mamba layers), whereas DynaGraph is **training-free and model-agnostic**.

Phase ablation (Table 7). On the heavily loaded Qwen3.5-4B, where FCFS completely collapses (0% SLO@500ms), neither Phase 2 nor Phase 3 alone suffices: C3 (P2 only) reduces avg K to 14.7 but cannot reorder the FCFS queue (0% SLO); B4 (P3 only) reorders requests but faces uniformly heavy K=20 prompts (57.8% SLO). Only C5 (P2+P3 joint) achieves 94.9% SLO—P2 creates cost heterogeneity ($K_{\text{easy}} \ll K_{\text{hard}}$) that P3 exploits via priority scheduling, while backpressure further compresses K_{eff} under sustained congestion. This synergy is the core architectural insight of DynaGraph and cannot be replicated by KV-cache optimization alone.

Table 8: Scheduling strategy comparison (2Wiki, Qwen3-4B TP1, Zipf@30qps, 1000q×3000req). Each method uses its natural K strategy.

Method	K strat.	Sched	F1↑	avg K	p50↓	p95↓	p99↓	$\mathcal{J}$@200↑	$\mathcal{J}$@500↑
C1 (Static)	Fixed 20	FCFS	.759	20.0	359	1560	4122	19.0%	54.3%
SJF K=20	Fixed 20	SJF	.756	20.0	344	2022	4228	21.3%	53.7%
EDF K=20	Fixed 20	EDF	.756	20.0	329	1034	5469	20.8%	60.3%
SJF K=10	Fixed 10	SJF	.742	10.0	133	343	1465	61.5%	78.0%
C3 (AdpK only)	Diff, no BP	FCFS	.750	14.7	181	501	3000	45.5%	76.5%
AdpK+BP	Diff+BP	FCFS	.755	12.5	173	476	2969	47.8%	77.4%
DynaGraph	Diff+BP	Comp.	.755	12.5	**143**	**427**	3310	**55.6%**	**77.9%**

5.4 RQ3: Scheduling Strategy Analysis

Table 8 compares scheduling strategies on Qwen3-4B (TP1, higher load pressure than TP2). Each method uses its natural K allocation to reflect realistic deployment.

Analysis. The table reveals a clear decomposition of contributions: **(1) Scheduling alone has limited effect.** With fixed K=20, SJF improves $\mathcal{J}$@200 by only +2.3pp over FCFS (21.3% vs 19.0%), and EDF by +1.8pp. When all prompts are uniformly heavy, the scheduler can reorder but cannot reduce total load—validating Obs. 3 (Section 3).

Table 9: Context aggregation comparison (2Wiki, Qwen3.5-4B TP2, Zipf@30qps, τ=2s). $\mathcal{J}$ = QA-Goodput (soft, F1-weighted).

Method	F1↑	avg K	$\mathcal{J}$@2s↑	p50↓	p99↓
Full K=20	.784	20.0	61.0%	305	9127
Uniform K=10	.780	10.0	60.6%	223	1740
Adaptive − K [40]	.788	11.2	59.5%	229	3220
CompSelect [50]	–	–	–	–	–
RAG − CSM [13]	–	–	–	–	–
DynaGraph P2	**.796**	14.3	**87.4%**	255	2934

(2) Adaptive K is the dominant factor. C3 (difficulty-aware K, no backpressure) lifts $\mathcal{J}$@200 from 19.0% to 45.5% (+26.5pp), dwarfing any scheduling-only gain. The graph-topological difficulty scoring (Phase 2) is the primary lever.

(3) Backpressure adds headroom. Adding the P3→P2 backpressure loop (C3 → AdpK+BP) further improves $\mathcal{J}$@200 from 45.5% to 47.8% (+2.3pp) by dynamically compressing K_{eff} under congestion.

(4) Composite priority improves median and tail latency. DynaGraph's multi-signal priority adds +7.8pp $\mathcal{J}$@200 over FCFS with the same adaptive K (55.6% vs 47.8%), while reducing p50 from 173 ms to 143 ms (−17%) and p95 from 476 ms to 427 ms (−10%).

(5) Brute-force K reduction is suboptimal. SJF + K=10 achieves the highest $\mathcal{J}$@200 (61.5%) by aggressively shortening all prompts, but at the cost of the lowest F1 (0.742 vs DynaGraph 0.755). DynaGraph preserves quality through *selective* budget allocation: hard queries retain large K while easy ones are compressed.

5.5 RQ4: Context Aggregation Comparison

Table 9 compares context selection methods with fixed FCFS scheduling to isolate Phase 2's contribution.

Key findings. (1) Adaptive − K [40] thresholds on similarity scores, reducing avg K to 11.2 but achieving only 59.5% GP—comparable to Uniform K=10 (60.6%) because it lacks load-aware backpressure. (2) DynaGraph P2 achieves 87.4% GP (+26pp over Adaptive − K) with higher F1 (0.796 vs 0.788). The gain comes from difficulty-aware allocation (graph-topological features) combined with serving-aware adaptation (backpressure). (3) Random $K \in [5, 20]$ allocation (tested on Qwen3-4B TP1) collapses F1 to 0.357 (−0.40 vs DynaGraph), confirming that S_{diff}'s difficulty-aware allocation is essential.

Comparison with trained methods. CompSelect [50] requires per-dataset SFT; RAG − CSM [13] trains a learned surrogate model. Both are orthogonal to DynaGraph's training-free approach and could serve as drop-in replacements for the scoring function, though their serving integration remains unexplored.

5.6 RQ5: Phase 2 Design Choices

Figure 8 ablates three design decisions in Phase 2, using 2Wiki on Qwen3.5-4B at Zipf@30qps.

Design analysis. (a) **Decay shape.** Exponential decay achieves the best GP@2s and preserves difficulty differentiation under congestion. At τ=200 ms, Exponential achieves $\mathcal{J}_\lambda$=35.7% vs. Linear 37.8%, Fixed 32.0%, and Threshold 27.0%. Linear outperforms Exponential only at the strictest SLO (τ=200 ms); Exponential overtakes beyond $\tau \approx 1$ s (75.4% vs. Linear 74.8%) because it preserves

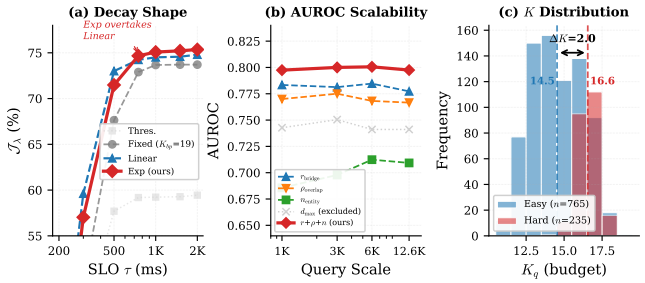


Figure 8: Phase 2 design ablation (2Wiki, Qwen3.5-4B, λ=30). (a) Exp decay achieves best GP@2s (75.4%); Linear leads only at τ=200 ms. (b) Three features (r+ρ+n) achieve stable AUROC ≈ 0.80; $d_{\max}$ excluded. (c) Hard/Easy K separation: $\Delta\bar{K}$=2.0.

a wider K spread under sustained load ($\bar{K}$=12.5 vs. 10.1), validating Theorem 3's optimality. Fixed (K_{bp} constant) achieves decent GP@500 (67.6%) but its P99 reaches **15.7 s**—the rigid cap cannot absorb load surges. Threshold (Step3) collapses K to $K_{\min}$ uniformly, eliminating difficulty awareness entirely (GP@2s = 59.4%).

(b) **Feature selection.** r_{bridge} alone achieves the highest single-feature AUROC (≈0.79); ρ_{overlap} follows at ≈0.77; n_{entity} contributes modestly (≈0.70). The three-feature combination reaches AUROC = 0.80, while adding $d_{\max}$ (≈0.74) yields no marginal gain, confirming three features as the effective ceiling. All features remain stable from 1K to 12.6K queries, indicating that the topological signals generalize without per-scale recalibration. This also justifies Phase 1's bounded two-hop expansion: deeper traversal would discover additional high-degree entities, but degree alone ($d_{\max}$) carries minimal predictive signal.

(c) K **distribution.** Adaptive K produces well-separated allocations: Hard queries concentrate at $\bar{K}$=16.6, Easy queries at $\bar{K}$=14.5 ($\Delta\bar{K}$=2.0). The observed range $[12.5, 19]$ is narrower than the theoretical $[K_{\min}=10, K_{\max}=20]$, confirming that backpressure actively compresses budgets under 30 QPS load while still preserving difficulty differentiation.

Hyperparameter robustness. $K_{\min}$ and θ_{bp} are robust: $\mathcal{J}$ varies by <2pp across $K_{\min} \in \{5, 8, 10\}$ and $\theta_{\text{bp}} \in [0.3, 0.9]$ (Appendix Tables **??** and **??**).

5.7 RQ6: Load Sensitivity and Runtime Behavior

Runtime behavior. Figure 9 visualizes the closed-loop mechanism under Zipf@5QPS. The system operates in quality-first mode 69% of the time; during load spikes, $K_{\text{eff}}(t)$ decays exponentially, compressing budgets while preserving difficulty differentiation.

Load sensitivity. A QPS sweep from 2 to 30 (Appendix Table **??**) confirms that C5 consistently improves SLO@500ms by 7–12pp with stable GP@500 gain (~8pp) across all arrival rates.

5.8 RQ7: End-to-End Deployment

We validate DynaGraph under a fully online pipeline where retrieval runs on GPU 0 (embedding + FAISS + graph expansion + reranking) and LLM serving runs on GPU 1.

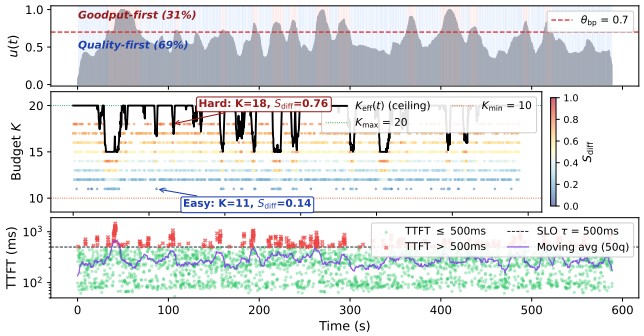


Figure 9: Runtime trace of closed-loop budget allocation (2Wiki, Qwen3.5-4B, Zipf@5QPS, 3000 requests, $\theta_{\mathbf{bp}}$=0.7). Top: Queue utilization $u(t)$; quality-first mode 69% of the time (blue), goodput-first during spikes (red). Middle: $K_{\mathbf{eff}}(t)$ (black curve) decays exponentially under congestion; per-query K_q (dots) spread within $[K_{\min}, K_{\mathbf{eff}}(t)]$—even during congestion, hard queries retain K=18 while easy ones compress to K=11. Bottom: Per-query TTFT (log scale); SLO violations cluster after sustained bursts, consistent with Theorem 6.

Table 10: End-to-end deployment (1000q, 5QPS, ~6K corpus). E2E TTFT = retrieval + generation.

Dataset	Latency (ms) Retr.↓	Gen.↓	E2E↓	p95↓	p99↓	Quality F1↑	Acc↑
Qwen3.5-4B							
2Wiki	178	414	613	1157	12611	.797	.842
HotpotQA	270	231	529	724	824	.714	.670
MuSiQue	300	339	679	922	1349	.399	.421
Qwen3-4B							
2Wiki	180	304	502	888	1054	.764	.803
HotpotQA	254	138	395	525	602	.698	.650
MuSiQue	352	276	614	836	978	.371	.341

Table 11: E2E corpus scalability (2Wiki nested subsets, Qwen3.5-4B, 3QPS).

Queries	Corpus	Retr.↓	E2E↓	p99↓	F1↑	Acc↑
1,000	6K	178	613	12611	.797	.842
3,000	18K	158	515	1041	.596	.673
6,000	32K	162	510	1040	.597	.666
12,576	57K	208	642	1168	.686	.770

Cross-dataset E2E. Table 10 reports end-to-end TTFT on three datasets under two models at 5 QPS.

Observations. (1) Retrieval overhead varies by dataset: MuSiQue requires 300–352 ms (larger subgraphs) vs 2Wiki's 178–180 ms. (2) E2E TTFT remains under 700 ms across all settings, with Qwen3-4B achieving p50<620 ms on all datasets. (3) F1 in E2E matches the LLM-only numbers (e.g., 2Wiki E2E F1=0.797 vs LLM-only F1=0.787 on Qwen3.5-4B), confirming that online retrieval does not degrade quality.

Corpus scalability. Table 11 evaluates DynaGraph as corpus size grows from 6K to 57K passages (12×).

Scalability analysis. (1) **E2E TTFT is stable across 10× corpus scaling**: p50 ranges from 510–642 ms despite corpus growing from 6K to 57K passages. (2) Retrieval overhead increases modestly: FAISS search grows from 4 ms (6K) to 32 ms (57K)—the embedding (26 ms) and reranking (118 ms) stages remain constant since they process a fixed number of candidates per query. (3) F1 drops from 0.797 to 0.597 at 3× scale due to dense retrieval recall dilution (larger corpus → more distractors), then recovers to 0.686 at 12× where the full development set has better query-document coverage. This recall issue is inherent to dense retrieval and shared by all RAG systems; DynaGraph's serving optimization (Phase 2+3) remains effective regardless of corpus scale.

REFERENCES

[1] Amey Agrawal, Nitin Kulkarni, et al. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[2] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. 2018. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books. https://pages.cs.wisc.edu/~remzi/OSTEP/

[3] Albert-László Barabási and Réka Albert. 1999. Emergence of Scaling in Random Networks. *Science* 286, 5439, 509–512.

[4] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. 2016. *Site Reliability Engineering: How Google Runs Production Systems.* " O'Reilly Media, Inc.".

[5] Angela Bonifati, Wim Martens, John Timm, et al. 2017. An Analytical Study of Large SPARQL Query Logs. *Proceedings of the VLDB Endowment* 11, 2 (2017), 149–161.

[6] Lee Breslau, Pei Cao, Li Fan, Graham Phillips, and Scott Shenker. 1999. Web caching and Zipf-like distributions: Evidence and implications. In *Proceedings of IEEE INFOCOM*. IEEE.

[7] Niv Buchbinder and Joseph Naor. 2009. *The design of competitive online algorithms via a primal-dual approach*. Now Publishers.

[8] Deeparnab Chakrabarty, Yunhong Zhou, and Rajan Lukose. 2008. Online knapsack problems. In *Workshop on Internet and Network Economics (WINE)*.

[9] Yixiao Chen et al. 2025. Mitigating Hallucination in Large Language Models: An Application-Oriented Survey on RAG, Reasoning, and Agentic Systems. *arXiv preprint arXiv:2510.24476* (2025). Section IV.B discusses overthinking in reasoning-augmented LLMs.

[10] Alla Chepurova, Aydar Bulatov, Mikhail Burtsev, and Yuri Kuratov. 2026. Wikontic: Constructing Wikidata-Aligned, Ontology-Aware Knowledge Graphs with Large Language Models. In *EACL*. 8304–8319. https://doi.org/10.18653/v1/2026.eacl-long.388

[11] Yingnong Dang, Qingwei Lin, and Peng Huang. 2019. AIOps: Real-World Challenges and Research Innovations. In *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering: Companion (ICSE-Companion)*. 4–5.

[12] George B. Dantzig. 1957. Discrete-Variable Extremum Problems. *Operations Research* 5, 2 (1957), 266–288.

[13] Jiale Deng, Yanyan Shen, Ziyuan Pei, Youmin Chen, and Linpeng Huang. 2025. Influence Guided Context Selection for Effective Retrieval-Augmented Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2509.21359.

[14] Darren Edge et al. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130* (2024).

[15] Jack Edmonds. 1971. Matroids and the Greedy Algorithm. *Mathematical Programming* 1, 1 (1971), 127–136.

[16] Teofilo F. Gonzalez. 1985. Clustering to Minimize the Maximum Intercluster Distance. *Theoretical Computer Science* 38 (1985), 293–306.

[17] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[18] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[19] Joseph L. Hellerstein, Yixin Diao, Sujay Parekh, and Dawn M. Tilbury. 2004. *Feedback Control of Computing Systems*. John Wiley & Sons. https://doi.org/10.1002/047166880X Foundational reference for applying feedback control theory to computing systems; cited for the backpressure loop analogy.

[20] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A Multi-Hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *COLING*.

[21] Lei Huang et al. 2025. Large Language Models Hallucination: A Comprehensive Survey. *arXiv preprint arXiv:2510.06265* (2025).

[22] Isaacus. 2025. semchunk: A fast, lightweight and easy-to-use Python library for splitting text into semantically meaningful chunks. https://github.com/isaacus-dev/semchunk

[23] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. 7036–7050.

[24] Wenqi Jiang et al. 2025. RAGO: Systematic Performance Optimization for Retrieval-Augmented Generation Serving. In *ISCA*.

[25] Yunxiang Jiang, Zhiliang Luo, Lei Li, and Yongfeng Zhang. 2025. Clue-RAG: Towards Accurate and Cost-Efficient Graph-based RAG via Multi-Partite Graph and Query-Driven Iterative Retrieval. *arXiv preprint arXiv:2507.08445* (2025).

[26] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. 2024. RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation. *arXiv preprint arXiv:2404.12457* (2024).

[27] Kostis Kaffes, Timothy Chong, Jack T. Humphries, Adam Belay, David Mazières, and Christos Kozyrakis. 2019. Shinjuku: Preemptive Scheduling for μsecond-scale Tail Latency. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[28] Richard M. Karp. 1972. Reducibility among Combinatorial Problems. In *Complexity of Computer Computations*. Plenum Press, 85–103.

[29] Bernhard Korte and Dirk Hausmann. 1978. An Analysis of the Greedy Heuristic for Independence Systems. In *Annals of Discrete Mathematics*. Vol. 2. Elsevier, 65–74.

[30] Woosuk Kwon et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*.

[31] Patrick Lewis et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.

[32] Costas Mavromatis and George Karypis. 2024. GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning. *arXiv preprint arXiv:2405.20139* (2024).

[33] Sean P. Meyn and Richard L. Tweedie. 2009. *Markov Chains and Stochastic Stability* (2nd ed.). Cambridge University Press.

[34] Michael J. Neely. 2010. *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Morgan & Claypool Publishers. Establishes that queue backlogs serve as Lagrange multipliers in the Lyapunov drift-plus-penalty framework.

[35] Xinyuan Ni, Jianwei Xu, Yi Liu, Hongtao Chen, and Lu Wang. 2025. KET-RAG: A Cost-Efficient Multi-Granular Indexing Framework for Graph-RAG. *arXiv preprint arXiv:2502.09304* (2025).

[36] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2024. Unifying Large Language Models and Knowledge Graphs: A Roadmap. *IEEE Transactions on Knowledge and Data Engineering* (2024).

[37] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 conference on empirical methods in natural language processing*. 2383–2392.

[38] Siddhant Ray, Rui Pan, Zhuohan Gu, Kuntai Du, Shaoting Feng, Ganesh Ananthanarayanan, Ravi Netravali, and Junchen Jiang. 2025. METIS: Fast Quality-Aware RAG Systems with Configuration Adaptation. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*. 606–622.

[39] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. 2018. *Operating System Concepts* (10th ed.). Wiley.

[40] Chihiro Taguchi, Seiji Maekawa, and Nikita Bhutani. 2025. Efficient Context Selection for Long-Context QA: No Tuning, No Iteration, Just Adaptive-k. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

[41] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single Hop Question Composition. *TACL* (2022).

[42] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: A Free Collaborative Knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85.

[43] Jiahao Wang et al. 2026. From Prefix Cache to Fusion RAG Cache: Accelerating LLM Inference in Retrieval-Augmented Generation. *arXiv preprint arXiv:2601.12904* (2026).

[44] Lei Yang, Mohammad Hajiesmaili, Ramesh Sitaraman, and Don Towsley. 2022. Competitive algorithms for online multidimensional knapsack problems. In *Proc. ACM SIGMETRICS*.

[45] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *EMNLP*.

[46] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the twentieth European conference on computer systems*. 94–109.

[47] Hao Yuan et al. 2026. DepCache: A KV Cache Management Framework for GraphRAG with Dependency Attention. In *Proceedings of the ACM on Management of Data (SIGMOD)*.

[48] Bowen Zhang and Harold Soh. 2024. Extract, Define, Canonicalize: An LLM-Based Framework for Knowledge Graph Construction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 9820–9836. Ontology-constrained NER+entity resolution framework; referenced for grounding entity extraction against predefined schemas.

[49] Nan Zhang, Prafulla Kumar Choubey, Alexander Fabbri, Gabriel Bernadett-Shapiro, Rui Zhang, Prasenjit Mitra, Caiming Xiong, and Chien-Sheng Wu. [n.d.]. SiReRAG: Indexing Similar and Related Information for Multihop Reasoning. In *The Thirteenth International Conference on Learning Representations*.

[50] Qianchi Zhang, Hainan Zhang, Liang Pang, Yongxin Tong, Hongwei Zheng, and Zhiming Zheng. 2025. Less is More: Compact Clue Selection for Efficient Retrieval-Augmented Generation Reasoning. *arXiv preprint arXiv:2502.11811* (2025).

[51] Lianmin Zheng et al. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *NeurIPS*.

[52] Yinmin Zhong, Shengyu Liu, Jianbo Chen, et al. 2024. DistServe: Disaggregating

Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. Formalizes TTFT, TPOT, and Goodput in LLM context.

[53] Yunhong Zhou, Deeparnab Chakrabarty, and Rajan Lukose. 2008. Budget optimization for online advertising.

A PROOFS OF THEORETICAL RESULTS

This appendix provides complete proofs for the theoretical results distributed throughout Section 4.5.1–Section 4.5.3.

A.1 Proof of Theorem 2 (NP-Hardness)

Proof. We reduce from the NP-complete Partition problem [28].

Source problem (Partition). Given a multiset $A = \{a_1, \ldots, a_m\}$ of positive integers with $\sum_{j=1}^{m} a_j = 2B$, decide whether there exists a subset $A' \subseteq A$ such that $\sum_{j \in A'} a_j = B$.

Reduction. Given a Partition instance A, construct a QA-Goodput scheduling instance with query set $\mathcal{S} = \{q_1, \ldots, q_m\}$ and fixed budget vector K:

- Each K_i is chosen such that $L_i(K_i)/\mu = a_i$ (the prefill time of q_i equals a_i);
- $\tau_{q_i} = B$ for all i (common deadline);
- $Q(K_i, q_i) = a_i$ (quality proportional to processing time);
- $\eta = B$ (threshold).

Correctness. ($\Rightarrow$) Suppose a partition A' exists with $\sum_{j \in A'} a_j = B$. Schedule the corresponding queries first in any order. Their total processing time is $\sum_{i:q_i \in A'} L_i(K_i)/\mu = B \le \tau_{q_i}$, so all complete by the deadline. The QA-Goodput is $\sum_{i \in A'} Q(K_i, q_i) = B \ge \eta$.

($\Leftarrow$) Suppose a serving order π achieves QA-Goodput $\ge \eta = B$. Let S be the set of on-time queries. Then $\sum_{i \in S} Q(K_i, q_i) = \sum_{i \in S} a_i \ge B$. Since all on-time queries complete by deadline B, $\sum_{i \in S} L_i(K_i)/\mu = \sum_{i \in S} a_i \le B$. Combining: $\sum_{i \in S} a_i = B$, so S corresponds to a valid partition.

Since Partition is NP-complete, the restricted decision problem is NP-hard. Since Eq. 3 additionally optimizes over K, it is at least as hard. □

A.2 Proof of Theorem 5 (Scheduling Approximation)

Proof. The proof establishes both parts of the theorem.

Part (i): Knapsack bound (1/2).

Step 1 — Knapsack reduction. With common deadline τ, a subset S of queries is feasibly schedulable if and only if $\sum_{i \in S} p_i \le \tau$ (non-preemptive, single machine, any ordering within S works as long as the total fits). This is exactly the 0-1 knapsack problem:

$$\max \sum_i v_i x_i \quad \text{s.t.} \quad \sum_i p_i x_i \le \tau, \quad x_i \in \{0, 1\}.$$

Step 2 — Fractional relaxation. Relaxing x_i to $[0, 1]$, the fractional optimum V_{frac} is obtained by sorting items in decreasing value-density v_i/p_i and greedily filling the knapsack [12]. Since the integer optimum satisfies $\text{OPT} \le V_{\text{frac}}$, this provides an upper bound.

Step 3 — Greedy gap analysis. Consider the greedy integral packing in the same v_i/p_i order. Let j be the first item that does not fit. The greedy packing value satisfies $V_{\text{pack}} \ge V_{\text{frac}} - v_j$, because the fractional solution assigns j a fractional value of at most v_j. Meanwhile, the maximum single-item value $v_{\max} = \max_i v_i \ge v_j$.

Step 4 — Combine. Taking the better of the greedy packing and the best single item:

$$V_{\text{greedy}} = \max(V_{\text{pack}},\ v_{\max}) \ge \max(\text{OPT} - v_j,\ v_j) \ge \frac{\text{OPT}}{2},$$

where the last inequality holds because for any $a, b \ge 0$ with $a+b \ge \text{OPT}$, $\max(a, b) \ge \text{OPT}/2$. No assumption on the distribution of processing times $\{p_i\}$ is required.

Part (ii): Synergy bound via $r(t)$-independence system.

Step 1 — Processing-time ratio. Phase 2's budget allocation constrains $K_i \in [K_{\min}, K_{\text{eff}}(t)]$. Since $p_i \propto K_i$, the ratio of maximum to minimum processing time within a scheduling window is:

$$r(t) = \frac{\max_i p_i}{\min_i p_i} \le \frac{K_{\text{eff}}(t)}{K_{\min}}.$$

Step 2 — Independence system structure. With heterogeneous processing times, the family of feasibly schedulable subsets (under common deadline τ) forms an *independence system* $(Q, \mathcal{I})$ but not necessarily a matroid. We show it is an $r(t)$-system: for any element $q \in Q$, the ratio of cardinalities of any two maximal independent sets containing q is at most $r(t)$. This holds because the largest feasible set packs at most $\lfloor \tau/p_{\min} \rfloor$ items, while any feasible set containing the largest item has at most $\lfloor \tau/p_{\max} \rfloor + 1$ items.

Step 3 — Greedy guarantee. By the Jenkyns–Korte–Hausmann theorem [29], greedy by value-density on a p-independence system with modular (additive) objective achieves:

$$\mathcal{J}_{\text{greedy}} \ge \frac{1}{1 + r(t)} \cdot \text{OPT}.$$

Step 4 — Load-dependent improvement. Since $K_{\text{eff}}(t)$ is monotonically decreasing in $u(t)$ (Eq. 8), $r(t)$ decreases as load increases:

$$r(t) = \frac{K_{\text{eff}}(t)}{K_{\min}} \xrightarrow{u(t) \to 1} \frac{K_{\min}}{K_{\min}} = 1.$$

When $r(t) = 1$, all processing times are equal, the feasible set family satisfies the matroid exchange axiom [15], and greedy by value achieves the exact optimum. □

A.3 Proof of Theorem 6 (Queue Stability)

Proof. We model the system as a single-server queue with state-dependent service times and Markov-modulated arrivals, using a slotted-time model with slot duration δ.

System model. Let $Q(t)$ be the number of requests in the queue at the beginning of slot t. Arrivals follow an MMPP: a background Markov chain $\{Z(t)\}$ with finite state space $\mathcal{Z}$ modulates the arrival rate, producing $A(t) \sim \text{Poisson}(\lambda_{Z(t)}\delta)$ arrivals per slot. Define $\lambda_{\max} = \max_{z \in \mathcal{Z}} \lambda_z$.

The service time for query q in slot t depends on the backpressure state. Under Phase 2's controller (Eq. 8), the per-query budget satisfies $K_q \le K_{\text{eff}}(t)$. Under sustained congestion ($u(t) \gg \theta_{\text{bp}}$), $K_{\text{eff}}(t) \to K_{\min}$, so the worst-case service time is bounded by $K_{\min}/\mu$.

Lyapunov function. Define $V(t) = \frac{1}{2}Q(t)^2$. The one-slot conditional drift is:

$$\Delta V(t) \triangleq \mathbb{E}\big[V(t+1) - V(t) \mid Q(t), Z(t)\big]. \tag{18}$$

Using the dynamics $Q(t+1) = \max(Q(t) - D(t), 0) + A(t)$:

$$\Delta V(t) \le Q(t) \cdot \big(\mathbb{E}[A(t) \mid Z(t)] - \mathbb{E}[D(t) \mid Q(t)]\big) + B, \qquad (19)$$

where $B = \frac{1}{2}\big((\lambda_{\max}\delta)^2 + (\mu\delta/K_{\min})^2\big)$ bounds the second-order terms.

Service rate under backpressure. When $Q(t)$ is large (more precisely, when $u(t) > \theta_{\text{bp}}$), Phase 2 drives $K_{\text{eff}}(t) \to K_{\min}$, so every query's service time is bounded by $K_{\min}/\mu$. The effective service rate per slot is at least $\mu\delta/K_{\min}$ queries.

Negative drift. Substituting into Eq. 19:

$$\Delta V(t) \le Q(t) \cdot \delta\left(\lambda_{\max} - \frac{\mu}{K_{\min}}\right) + B. \qquad (20)$$

When $\lambda_{\max} \cdot K_{\min}/\mu < 1$ (Eq. 17), define:

$$\epsilon \triangleq \frac{\mu}{K_{\min}} - \lambda_{\max} > 0.$$

Then $\Delta V(t) \le -\epsilon\delta \cdot Q(t) + B$, which is negative for $Q(t) > B/(\epsilon\delta)$.

Foster–Lyapunov criterion. By the Foster–Lyapunov theorem [33], the existence of a Lyapunov function with negative drift outside a finite set $\{Q : Q \le B/(\epsilon\delta)\}$ implies positive recurrence of the joint process $(Q(t), Z(t))$. Since $\{Z(t)\}$ is ergodic by assumption and the drift condition holds uniformly over all states $z \in \mathcal{Z}$ (because we use peak rate $\lambda_{\max}$), $Q(t)$ is positive recurrent, i.e., $\mathbb{E}[Q] < \infty$.

Quantitative queue-length bound. By the quantitative Foster–Lyapunov theorem [34], taking expectations over the stationary distribution:

$$\mathbb{E}[Q] \le \frac{B}{\epsilon\delta}. \qquad (21)$$

This bound is used in the starvation analysis (Appendix A.4). □

A.4 Proof of Corollary ?? (Bounded Starvation)

Proof. Consider a request q that enters the queue at time t_0 with base priority $\mathcal{P}(q)$. We show that q is served within bounded expected time via a three-step argument.

Step 1: Domination horizon. From Eq. 4.5.3, the aged priority at elapsed wait time $t_{\text{wait}}(q) = t - t_0$ is:

$$\mathcal{P}_{\text{aged}}(q) = \mathcal{P}(q) + \Delta_{\text{age}} \cdot \max\left(0,\ \frac{t_{\text{wait}}(q) - T_{\text{age}}}{\tau_q}\right).$$

Every newly arriving request q' starts with $\mathcal{P}_{\text{aged}}(q') = \mathcal{P}(q') \le \mathcal{P}_{\max}$ and zero aging bonus. At time $t_0 + T_{\text{dom}}(q)$, where

$$T_{\text{dom}}(q) = T_{\text{age}} + \tau_q \cdot \frac{\mathcal{P}_{\max}}{\Delta_{\text{age}}},$$

the aging bonus of q equals $\mathcal{P}_{\max}$, so $\mathcal{P}_{\text{aged}}(q) \ge \mathcal{P}(q) + \mathcal{P}_{\max} \ge \mathcal{P}_{\max}$. Hence, from time $t_0 + T_{\text{dom}}(q)$ onward, q has priority at least as high as any fresh arrival.

Step 2: Expected queue-length bound. From the Lyapunov drift analysis in the proof of Theorem 6 (Eq. 21):

$$\mathbb{E}[Q] \le \frac{B}{\epsilon\delta},$$

where $B = \frac{1}{2}\left[(\lambda_{\max}\delta)^2 + (\mu\delta/K_{\min})^2\right]$ and $\epsilon = \mu/K_{\min} - \lambda_{\max} > 0$.

Step 3: Predecessor drain time. At time $T_{\text{dom}}(q)$, the requests still ahead of q in the priority queue are those that arrived before q with higher initial priority. By the same aging argument, these predecessors have even higher aged priority and will be served before q. Their expected count is bounded by $\mathbb{E}[Q]$ (the stationary queue length). Each predecessor requires at most $K_{\max}/\mu$ service time (the worst case occurs in quality-first mode where $K_q = K_{\max}$). Therefore:

$$T_{\text{drain}} \le \mathbb{E}[Q] \cdot \frac{K_{\max}}{\mu} = \frac{B}{\epsilon\delta} \cdot \frac{K_{\max}}{\mu}.$$

Combining. The expected total wait time for q is bounded by:

$$\mathbb{E}[W_q] \le T_{\text{dom}}(q) + T_{\text{drain}} = \left(T_{\text{age}} + \tau_q \cdot \frac{\mathcal{P}_{\max}}{\Delta_{\text{age}}}\right) + \frac{B}{\epsilon\delta} \cdot \frac{K_{\max}}{\mu}.$$

Both terms are finite when the stability condition (Eq. 17) holds ($\epsilon > 0$), completing the proof. □

A.5 Proof of Theorem 3 (Online Budget Selection)

We provide the full proof for completeness; the result synthesizes classical analyses from online knapsack theory.

Proof. **Part (a): Lower bound.** Consider a single knapsack of capacity C and items with value-to-cost ratio in $[L, U]$. An adversary constructs two input sequences: σ_1 consists of items with ratio L filling the knapsack, followed by items with ratio U; σ_2 consists only of items with ratio L. Any deterministic algorithm that reserves capacity for high-value items (performing well on σ_1) loses on σ_2, and vice versa. Balancing across all such adversarial constructions yields the lower bound $\ln(U/L) + 1$ [53].

Part (b): Optimality of the exponential threshold. Define the threshold function $\Psi : [0, 1] \to [L, U]$ as:

$$\Psi(z) = L \cdot \left(\frac{U}{L}\right)^z, \qquad (22)$$

where $z \in [0, 1]$ is the fraction of capacity consumed. The algorithm admits item i with ratio v_i if and only if $v_i \ge \Psi(z)$.

We verify the competitive ratio via the primal-dual method of Chakrabarty et al. [8]. Assign a dual variable $\psi(z) = \Psi(z)$ to the capacity constraint. For any admitted item i at utilization z_i, the primal gain is $v_i \ge \Psi(z_i)$. Integrating the dual over the utilization trajectory:

$$\int_0^Z \Psi(z)\, dz = \int_0^Z L \cdot \left(\frac{U}{L}\right)^z dz = \frac{L}{\ln(U/L)}\left[\left(\frac{U}{L}\right)^Z - 1\right],$$

where Z is the final utilization. The ratio of primal value to dual cost is bounded by $\ln(U/L) + 1$ for all input sequences, matching the lower bound from part (a).

Part (c): Sub-optimality of the linear threshold. A linear threshold $\Psi_{\text{lin}}(z) = L + (U - L) \cdot z$ admits items with ratio in $[L, L + (U-L)z]$ at utilization z. Under an adversarial arrival sequence that concentrates high-value items ($v_i \approx U$) at $z \to 1$, the algorithm has already consumed capacity on medium-value items, yielding a competitive gap of $\Theta(\sqrt{U/L})$. Yang et al. [44] provide the tight analysis for multidimensional knapsack, which specializes to the single-dimension case here. □

Mapping to DynaGraph. In our setting, each query q is an "item" with value $\text{Score}(K_q, q)$ and cost proportional to K_q. The ratio range $[L, U]$ corresponds to $[K_{\min}, K_{\max}]$: a hard query utilizing $K_{\max}$ achieves high value per token, while an easy query receiving $K_{\max}$ wastes most tokens. DynaGraph's effective ceiling (Eq. 8) instantiates Ψ in budget space:

$$K_{\text{eff}}(u) = K_{\min} + (K_{\max} - K_{\min}) \cdot \exp\left(-\ln \frac{K_{\max}}{K_{\min}} \cdot \frac{u - \theta_{\text{bp}}}{1 - \theta_{\text{bp}}}\right),$$

which is monotonically decreasing in u and bounded in $[K_{\min}, K_{\max}]$. The shift by θ_{bp} defines a free zone ($u \le \theta_{\text{bp}}$) where capacity is unconstrained, following the slack extension of Buchbinder and Naor [7].

Processing-Time Ratio. The budget decisions from Theorem 3 induce per-query processing times $p_i = L_i(K_i)/\mu$, where $K_i \in [K_{\min}, K_{\text{eff}}(t)]$. The ratio of maximum to minimum processing time within a scheduling window is therefore bounded by:

$$r(t) = \frac{K_{\text{eff}}(t)}{K_{\min}}.$$

Under quality-first mode ($u(t) \le \theta_{\text{bp}}$), $r(t) = K_{\max}/K_{\min}$. Under goodput-first mode ($u(t) \to 1$), $r(t) \to 1$. This quantity governs the scheduling approximation guarantee in Theorem 5 (Section 4.5.1).

Online vs. Offline Knapsack in DynaGraph. Phase 2 faces an *online* knapsack: queries arrive sequentially and the budget K_q must be irrevocably decided before observing future arrivals (Theorem 3). Phase 3, within a single scheduling window, faces an *offline* knapsack variant: all queued queries are visible and the scheduler selects which to serve within the deadline. The offline variant admits stronger guarantees (Theorem 5).

Compatibility with Theorem 6. The queue stability guarantee (Theorem 6) uses $K_{\min}$ as the worst-case budget under saturation. Since $K_{\text{eff}}(t)$ is monotonically decreasing and $K_{\text{eff}}(t) \to K_{\min}$ as $u(t) \to 1$, the Lyapunov argument applies directly with $K_{\min}$ replacing the original fixed K_{safe}.